

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN**



**Đề tài: Phát triển hệ thống Microservices
cho thương mại điện tử**

Giảng viên:

Kim Ngọc Bách

Nhóm lớp:

03

Nhóm bài tập:

11

Phạm Đức Toàn

B22DCCN735

Trương Huy Tâm

B22DCCN711

Lê Tuấn Phong

B22DCCN615

Lời cảm ơn

Nhóm chúng em xin gửi lời tri ân sâu sắc đến Quý Thầy/Cô tại Học viện Công nghệ Bru chính Viện thông nói chung và Khoa Công nghệ Thông tin nói riêng, vì đã tận tình giảng dạy và truyền đạt những nền tảng kiến thức quý báu trong suốt quá trình học tập. Đặc biệt, chúng em xin bày tỏ lòng biết ơn chân thành đến Thầy Kim Ngọc Bách – người đã trực tiếp hướng dẫn, đồng hành và hỗ trợ nhóm trong suốt quá trình thực hiện bài tập lớn này. Những lời chỉ bảo tận tâm, cùng phong thái làm việc nghiêm túc và tư duy nghiên cứu khoa học của Thầy không chỉ giúp nhóm hoàn thành đề tài mà còn là hành trang quý giá cho hành trình phát triển nghề nghiệp sau này. Kính chúc Thầy cùng Quý Thầy/Cô luôn dồi dào sức khỏe và gặt hái nhiều thành công trong sự nghiệp trồng người.

Mục Lục

Lời cảm ơn	1
Mục Lục	1
BÁO CÁO BÀI TẬP LỚN	3
Chương 1. TỔNG QUAN ĐỀ TÀI	3
1.1. Bối cảnh.....	3
1.2. Mục tiêu của đề tài	3
1.3. Phạm vi.....	4
Chương 2. CƠ SỞ LÝ THUYẾT VỀ HỆ THỐNG PHÂN TÁN	4
2.1. Kiến trúc Microservices	4
2.2. Mô hình giao tiếp giữa các dịch vụ.....	5
2.3. API Gateway.....	6
2.4. Giao tiếp đồng bộ: REST và gRPC	6
2.5. Giao tiếp bất đồng bộ: Message Broker và Event-driven Architecture.....	7
2.6. Mẫu Saga và quản lý transaction phân tán.....	8
2.7. Resilience: Retry, Circuit Breaker, Timeout, Bulkhead	9
2.8. Polyglot Persistence	10
2.9. Observability: Health Check, Distributed Tracing, Metrics	10
2.10. Background Jobs trong hệ thống phân tán	11
2.11. Containerization và Container Orchestration	12
2.12. Kubernetes: kiến trúc và các đối tượng cốt lõi.....	12
Chương 3. THIẾT KẾ HỆ THỐNG	15
3.1. Kiến trúc tổng quan.....	15
3.2. Danh sách microservice và trách nhiệm.....	16
3.3. API Gateway (Ocelot).....	17
3.4. Identity Provider (Duende IdentityServer).....	18
3.5. Các luồng giao tiếp tiêu biểu	19
3.6. Mô hình dữ liệu polyglot.....	23
Chương 4. CÀI ĐẶT VÀ TRIỂN KHAI.....	23
4.1. Cấu trúc dự án.....	23
4.2. Công nghệ sử dụng	24

4.3. Frontend (React SPA)	25
4.4. Triển khai bằng Docker Compose	26
4.5. Triển khai trên Kubernetes	26
Chương 5. KẾT QUẢ MINH HỌA	38
Chương 6. ĐÁNH GIÁ VÀ KẾT LUẬN	45
6.1. Những gì đã đạt được	45
6.2. Hạn chế.....	46
6.3. Hướng phát triển	46
6.4. Kết luận	47
TÀI LIỆU THAM KHẢO.....	47

BÁO CÁO BÀI TẬP LỚN

Chương 1. TỔNG QUAN ĐỀ TÀI

1.1. Bối cảnh

Các hệ thống thương mại điện tử hiện đại như Tiki, Shopee, Lazada đều có hàng trăm chức năng nghiệp vụ độc lập, lưu lượng truy cập rất lớn và yêu cầu nâng cấp liên tục. Kiến trúc monolith truyền thống nhanh chóng bộc lộ các điểm yếu: triển khai cứng nhắc, một module hỏng kéo theo toàn hệ thống, khó scale theo nghiệp vụ và khó áp dụng nhiều công nghệ lưu trữ khác nhau. Vì vậy, kiến trúc Microservices đã trở thành lựa chọn phổ biến, kéo theo nhu cầu hiểu và vận dụng các pattern của hệ thống phân tán.

1.2. Mục tiêu của đề tài

Xây dựng một hệ thống thương mại điện tử với đầy đủ các thành phần microservices tiêu biểu, bao gồm:

- Tách nghiệp vụ thành nhiều microservice độc lập (Product, Customer, Basket, Order, Inventory, Schedule Job).
- Áp dụng API Gateway làm điểm vào duy nhất, định tuyến và bảo vệ các service backend.
- Tích hợp Identity Provider chuẩn OAuth 2.0 / OpenID Connect cho xác thực phân tán.
- Sử dụng đồng thời nhiều mô hình giao tiếp: REST (HTTP/JSON), gRPC (HTTP/2 + Protobuf), và Message Broker (RabbitMQ + MassTransit) cho event-driven.
- Áp dụng Saga pattern để xử lý transaction phân tán ở luồng đặt hàng (Checkout).
- Vận hành nhiều loại cơ sở dữ liệu (polyglot persistence): MySQL, PostgreSQL, Redis, SQL Server, MongoDB.

- Tích hợp các thành phần phụ trợ: Hangfire (background jobs), Polly (resilience), Serilog + Elasticsearch + Kibana (centralized logging), Health Checks UI.
- Triển khai bằng Docker Compose, demo end-to-end qua giao diện web React.

1.3. Phạm vi

Đề tài tập trung vào việc thiết kế và cài đặt một hệ thống phân tán ở quy mô demo, vận dụng các nguyên lý và pattern phổ biến của kiến trúc microservices. Hệ thống bao gồm đầy đủ các thành phần cốt lõi: các dịch vụ nghiệp vụ độc lập, điểm vào thống nhất qua API Gateway, cơ chế xác thực và phân quyền tập trung, nhiều hình thức giao tiếp giữa các dịch vụ (đồng bộ và bất đồng bộ), quản lý transaction phân tán, cũng như giao diện người dùng kết nối hoàn chỉnh. Toàn bộ hệ thống được đóng gói và chạy trong môi trường container hóa cục bộ phục vụ mục đích học tập và minh họa.

Đề tài không đặt mục tiêu triển khai trên hạ tầng production thực tế, chưa giải quyết các yêu cầu về khả năng mở rộng ngang (horizontal scaling), chưa tích hợp các giải pháp giám sát và tracing phân tán ở mức enterprise, và không xây dựng các tính năng nghiệp vụ hoàn chỉnh như thanh toán thực hay quản lý vận chuyển.

- Frontend dùng OAuth 2.0 ROPC để đơn giản hóa demo, không áp dụng Authorization Code + PKCE như khuyến nghị cho SPA ngoài thực tế.

Chương 2. CƠ SỞ LÝ THUYẾT VỀ HỆ THỐNG PHÂN TÁN

2.1. Kiến trúc Microservices

Định nghĩa: Microservices là một phong cách kiến trúc, trong đó một ứng dụng được chia thành nhiều dịch vụ nhỏ, mỗi dịch vụ:

- Triển khai độc lập trong một tiến trình riêng.
- Sở hữu cơ sở dữ liệu riêng (database-per-service).
- Giao tiếp với các dịch vụ khác qua giao thức nhẹ.
- Được tổ chức quanh một bounded context nghiệp vụ.

- So sánh với Monolith. Monolith dễ phát triển ban đầu nhưng khó mở rộng và bảo trì khi hệ thống lớn. Microservices cho phép:
- Mở rộng từng dịch vụ độc lập theo tải.
- Mỗi nhóm phát triển sở hữu một dịch vụ riêng biệt: phát triển, triển khai, vận hành độc lập.
- Sử dụng công nghệ phù hợp cho từng nghiệp vụ (polyglot technology, polyglot persistence).
- Cô lập lỗi (fault isolation): một dịch vụ hỏng không kéo theo toàn hệ thống.

Chi phí: Microservices không miễn phí: kéo theo độ phức tạp trong vận hành (giám sát, triển khai nhiều dịch vụ), giao tiếp qua mạng (độ trễ, lỗi mạng), nhất quán dữ liệu phân tán (eventual consistency), bảo mật và truy vết.

Các nguyên lý cốt lõi: Single Responsibility, autonomous, decentralized data management, design for failure, infrastructure automation.

2.2. Mô hình giao tiếp giữa các dịch vụ

Giao tiếp giữa các dịch vụ trong hệ thống phân tán có thể phân loại theo hai trục độc lập:

- **Đồng bộ và bất đồng bộ:** Giao tiếp đồng bộ yêu cầu bên gửi chờ phản hồi từ bên nhận trong cùng một chu kỳ tương tác (request–response). Giao tiếp bất đồng bộ tách rời tạm thời (temporal decoupling) người gửi và người nhận: bên gửi chỉ phát đi thông điệp và tiếp tục công việc, không cần biết khi nào bên nhận xử lý.
- **Một–một và một–nhiều:** Một thông điệp có thể được định tuyến tới chính xác một bên nhận (point-to-point, command-style) hoặc phát rộng tới nhiều bên nhận cùng quan tâm (publish–subscribe, event-style).

Kết hợp hai trục cho ra bốn dạng giao tiếp cơ bản (đồng bộ một–một, đồng bộ một–nhiều, bất đồng bộ một–một, bất đồng bộ một–nhiều), trong đó dạng đồng bộ một–một và bất đồng bộ một–nhiều là phổ biến nhất.

Một hệ thống thực tế thường kết hợp nhiều dạng giao tiếp khác nhau: dùng giao tiếp đồng bộ cho luồng cần phản hồi tức thời, và giao tiếp bất đồng bộ cho luồng xử lý nền, luồng cần loose coupling, hoặc khi nhiều dịch vụ cùng phản ứng với một sự kiện.

2.3. API Gateway

Vấn đề: Khi hệ thống có nhiều dịch vụ, việc cho phép client (web, mobile) gọi trực tiếp từng dịch vụ gây ra nhiều bất lợi: client phải biết địa chỉ của mọi dịch vụ, phải tự xử lý các mối quan tâm xuyên suốt (xác thực, hạn chế tần suất, CORS), và mỗi lần tách hoặc gộp dịch vụ ở phía server đều phá vỡ phía client. Ngoài ra, để lộ cấu trúc nội bộ ra ngoài cũng gây rủi ro bảo mật.

Giải pháp: Đặt một **API Gateway** ở biên hệ thống, đóng vai trò:

- Điểm vào duy nhất (single entry point) cho mọi request từ ngoài.
- Định tuyến (routing) request đến đúng dịch vụ nội bộ.
- Xử lý các mối quan tâm xuyên suốt (cross-cutting concerns): xác thực, phân quyền, hạn chế tần suất, lưu đệm, ngắt mạch, ghi log, gộp response, tổng hợp tài liệu API.
- Cung cấp một giao diện API thân thiện với client, ẩn cấu trúc nội bộ phía sau.

Các tính năng nâng cao thường gặp ở gateway gồm: Backend-for-Frontend (BFF — tạo gateway riêng cho từng loại client), API composition (gộp kết quả từ nhiều dịch vụ thành một response), và protocol translation (chuyển đổi giao thức giữa client và service nội bộ).

2.4. Giao tiếp đồng bộ: REST và gRPC

REST (Representational State Transfer) là một phong cách kiến trúc xây dựng trên HTTP, trong đó tài nguyên được biểu diễn qua URL và thao tác qua các động từ HTTP (GET, POST, PUT, DELETE). Payload thường là văn bản dạng JSON. Ưu điểm:

- Đơn giản, được hỗ trợ rộng rãi ở mọi nền tảng và ngôn ngữ.
- Mềm dẻo về schema, dễ kiểm thử trực tiếp bằng trình duyệt hay các công cụ thông dụng.

- Tận dụng được cơ chế caching và proxy của HTTP.

Nhược điểm: payload dạng văn bản nên dài và chậm hơn dạng nhị phân; không hỗ trợ tốt streaming; không bắt buộc schema chặt nên khó đảm bảo tương thích phiên bản.

gRPC là khung làm việc cho Remote Procedure Call chạy trên HTTP/2, sử dụng định dạng nhị phân Protocol Buffers để mã hóa dữ liệu. Đặc điểm:

- Schema-first: dịch vụ và thông điệp được định nghĩa trong một tệp lược đồ, từ đó tự sinh mã nguồn cho cả client và server, đảm bảo tương thích kiểu chặt chẽ.
- Hiệu năng cao: payload nhị phân nhỏ, tận dụng multiplexing của HTTP/2.
- Hỗ trợ bốn kiểu RPC: unary (request–response), server streaming, client streaming, và bidirectional streaming.
- Phù hợp với giao tiếp service-to-service trong mạng nội bộ, nơi cần độ trễ thấp và lưu lượng cao.

Tiêu chí lựa chọn: REST thường được dùng cho API công khai và giao tiếp với client bên ngoài nhờ tính phổ biến và dễ tiêu thụ; gRPC phù hợp cho giao tiếp nội bộ giữa các dịch vụ, nơi yêu cầu hiệu năng và tính chặt chẽ về schema cao hơn.

2.5. Giao tiếp bất đồng bộ: Message Broker và Event-driven Architecture

Lý do sử dụng giao tiếp bất đồng bộ: Khi một dịch vụ chỉ cần *thông báo* tới dịch vụ khác mà không chờ phản hồi tức thời, hoặc khi cần nói lỏng liên kết (loose coupling), hoặc khi muốn nhiều dịch vụ cùng phản ứng với một sự việc, giao tiếp đồng bộ trở thành điểm yếu: nó làm gia tăng phụ thuộc thời gian và *lan truyền lỗi*. Nếu dịch vụ phía sau ngừng hoạt động, dịch vụ phía trước cũng bị tê liệt theo.

Message Broker là một thành phần trung gian, nhận thông điệp từ bên gửi (producer), lưu trữ tạm thời và chuyển tiếp tới bên nhận (consumer). Hai mô hình phân phối chính:

- **Queue (Point-to-Point):** mỗi thông điệp được tiêu thụ bởi *đúng một* consumer trong nhóm. Phù hợp với mô hình phân chia công việc.
- **Topic / Publish–Subscribe:** mỗi thông điệp được phát rộng tới *mọi* subscriber đang quan tâm. Phù hợp với mô hình thông báo sự kiện.

Khi sử dụng broker, lợi ích chính là decoupling ở ba khía cạnh: bên gửi và bên nhận không cần biết nhau (decoupling về định danh), không cần online cùng lúc (decoupling về thời gian), và không cần xử lý cùng tốc độ — broker đóng vai trò bộ đệm (decoupling về tốc độ).

Event-driven Architecture (EDA). Là phong cách kiến trúc lấy *sự kiện* làm trung tâm: thay vì các dịch vụ gọi nhau bằng câu lệnh trực tiếp (command), một dịch vụ phát ra sự kiện mô tả "một việc đã xảy ra" (ví dụ *Order Created*, *Payment Completed*), các dịch vụ khác lắng nghe và phản ứng theo cách riêng. Hệ quả của EDA:

- **Loose coupling rất mạnh:** bên phát sự kiện không cần biết ai sẽ tiêu thụ.
- **Khả năng mở rộng nghiệp vụ:** có thể thêm subscriber mới mà không cần sửa producer.
- **Eventual consistency:** thay vì đảm bảo nhất quán mạnh tức thời, hệ thống chấp nhận một khoảng thời gian ngắn không nhất quán đổi lấy tính sẵn sàng và khả năng chịu lỗi.

2.6. Mẫu Saga và quản lý transaction phân tán

Vấn đề: Trong kiến trúc monolith, một giao dịch nghiệp vụ (ví dụ đặt hàng) có thể được gói gọn trong một transaction ACID của một cơ sở dữ liệu duy nhất, đảm bảo *all-or-nothing*. Trong kiến trúc microservices, mỗi dịch vụ sở hữu cơ sở dữ liệu riêng, nên không thể trải dài một transaction qua nhiều dịch vụ. Có hai hướng giải quyết:

- **Two-Phase Commit (2PC):** chuẩn transaction phân tán cổ điển, yêu cầu một coordinator điều phối hai pha *prepare* và *commit*. Hạn chế: blocking, hiệu năng kém, đòi hỏi tất cả thành phần hỗ trợ giao thức, không phù hợp với hệ thống phân tán quy mô lớn và các kho dữ liệu hiện đại.
- **Saga:** chia giao dịch nghiệp vụ thành một chuỗi **transaction cục bộ** ở từng dịch vụ; nếu một bước thất bại, hệ thống chạy các **compensating transaction** ở các bước trước để hoàn nguyên ngữ nghĩa (semantic rollback).
- Saga có hai biến thể triển khai:
- **Choreography (vũ đạo):** không có thành phần điều phối trung tâm; mỗi dịch vụ tự lắng nghe sự kiện của các dịch vụ khác và quyết định bước tiếp theo. Ưu

điểm: đơn giản, loose coupling. Nhược điểm: khi luồng nghiệp vụ phức tạp, sự kiện đan xen nhau rất khó theo dõi và gỡ lỗi.

- **Orchestration (chỉ huy):** có một **orchestrator** đóng vai trò bộ điều khiển, gọi tuần tự từng dịch vụ và quyết định nhánh tiếp theo dựa trên kết quả. Ưu điểm: luồng tường minh, dễ kiểm soát rollback, dễ giám sát. Nhược điểm: orchestrator có thể trở thành điểm phụ thuộc trung tâm, cần được thiết kế cẩn thận.

Saga **không đảm bảo nhất quán mạnh**: trong khoảng thời gian saga chưa hoàn tất, dữ liệu giữa các dịch vụ có thể không nhất quán tạm thời. Hệ thống cần được thiết kế để chấp nhận **eventual consistency**, và mọi bước nghiệp vụ phải có hành động bồi hoàn tương ứng.

2.7. Resilience: Retry, Circuit Breaker, Timeout, Bulkhead

Mạng máy tính vốn không tin cậy: gói tin có thể bị mất, độ trễ tăng vọt, hoặc đầu nhận tạm thời quá tải. Mọi hệ thống phân tán phải đối mặt với khái niệm partial failure — chỉ một phần hệ thống gặp sự cố. Để hạn chế lan truyền lỗi, ngành công nghiệp đã đúc kết thành một bộ mẫu thiết kế kháng lỗi (resilience patterns):

- **Timeout:** giới hạn thời gian chờ phản hồi, ngăn luồng gọi bị "treo" vô hạn.
- **Retry:** thử lại với những lỗi có khả năng phục hồi (network glitch, lỗi tạm thời ở phía nhận). Cần kết hợp **exponential backoff** và **jitter** để tránh các lần thử lại đồng loạt làm sập tiếp dịch vụ phía sau.
- **Circuit Breaker:** lấy cảm hứng từ *cầu chì điện*. Khi dịch vụ phía sau liên tục lỗi, "cầu chì" được ngắt — các request mới bị từ chối ngay, không tiêu tốn tài nguyên chờ đợi. Sau một thời gian, hệ thống chuyển sang trạng thái thử lại có kiểm soát. Cơ chế này có ba trạng thái: *Closed* (bình thường), *Open* (đang ngắt), *Half-Open* (cho phép một số request thăm dò).
- **Bulkhead:** lấy hình ảnh từ *vách ngăn* trên tàu thủy. Tài nguyên (thread, connection) được chia thành các "ngăn" riêng cho từng dịch vụ phía sau, để một dịch vụ chậm không thể hút cạn tài nguyên của toàn bộ ứng dụng.
- **Fallback:** khi không thể lấy kết quả thật, trả về giá trị mặc định, dữ liệu cũ trong cache, hoặc một response suy biến — đảm bảo hệ thống vẫn phục vụ được phần nào người dùng.

Các mẫu này thường được kết hợp đồng thời (ví dụ: Timeout + Retry + Circuit Breaker bọc quanh cùng một lời gọi mạng) để tạo ra một tầng truy cập có khả năng tự bảo vệ.

2.8. Polyglot Persistence

Polyglot persistence là nguyên tắc cho phép mỗi dịch vụ lựa chọn loại cơ sở dữ liệu phù hợp nhất với đặc tính dữ liệu của domain mình, thay vì áp đặt một loại cơ sở dữ liệu duy nhất cho toàn hệ thống. Đây là hệ quả tự nhiên của nguyên lý database-per-service trong microservices: vì các dịch vụ vốn đã sở hữu cơ sở dữ liệu độc lập, không có lý do nào bắt buộc chúng phải dùng cùng một công nghệ lưu trữ.

Các họ cơ sở dữ liệu khác nhau có những điểm mạnh riêng:

- **Cơ sở dữ liệu quan hệ (RDBMS)**: dữ liệu có cấu trúc rõ ràng, ràng buộc toàn vẹn mạnh, hỗ trợ transaction ACID, phù hợp với nghiệp vụ cốt lõi.
- **Kho khóa-giá trị (Key-Value Store)**: truy cập cực nhanh theo khóa, phù hợp với dữ liệu tạm, giỏ hàng, cache, phiên đăng nhập.
- **Cơ sở dữ liệu tài liệu (Document Database)**: schema linh hoạt, lưu trữ tài liệu lồng nhau, phù hợp với dữ liệu log, dữ liệu bán cấu trúc.
- **Cơ sở dữ liệu cột (Column-family)** và **đồ thị (Graph)**: phù hợp với phân tích quy mô lớn và dữ liệu nhiều liên kết.

Đánh đổi của polyglot persistence là chi phí vận hành tăng: nhóm phát triển phải am hiểu nhiều công nghệ, hệ thống phải sao lưu/giám sát nhiều loại kho dữ liệu, và việc tổng hợp dữ liệu xuyên dịch vụ trở nên phức tạp hơn.

2.9. Observability: Health Check, Distributed Tracing, Metrics

Trong kiến trúc microservices, một request đầu vào có thể đi qua nhiều dịch vụ, nhiều cơ sở dữ liệu, qua cả các hàng đợi bất đồng bộ — khiến việc hiểu *điều gì đang xảy ra* trở nên khó khăn. **Observability** (khả năng quan sát) đề cập tới năng lực suy luận trạng thái nội tại của hệ thống chỉ thông qua các tín hiệu mà nó phát ra. Ba trụ cột chính trong hệ thống phân tán:

- **Health Check**: Mỗi dịch vụ phơi bày một endpoint chuẩn báo cáo trạng thái nội tại (kết nối cơ sở dữ liệu, hàng đợi, các phụ thuộc). Hạ tầng (load balancer,

hệ thống điều phối container) sử dụng tín hiệu này để quyết định một thực thể có còn "sống" và sẵn sàng phục vụ hay không, từ đó điều hướng lưu lượng hoặc khởi động lại.

- **Distributed Tracing:** Mỗi request được gắn một **trace ID** xuyên suốt, và mỗi đoạn xử lý ở mỗi dịch vụ là một **span**. Khi một dịch vụ gọi sang dịch vụ khác, trace ID được truyền theo header. Hệ thống tracing tổng hợp lại toàn bộ chuỗi span thành biểu đồ dạng *waterfall*, cho phép xác định chính xác đoạn nào tốn thời gian, đoạn nào lỗi. Đây là công cụ then chốt khi gỡ lỗi hiệu năng trong hệ thống nhiều dịch vụ.
- **Metrics:** Là các số liệu định lượng được thu thập theo thời gian (số request mỗi giây, độ trễ phân vị thứ 95, tỷ lệ lỗi, tải CPU, dung lượng hàng đợi...). Phù hợp để thiết lập **cảnh báo** và quan sát xu hướng.

2.10. Background Jobs trong hệ thống phân tán

Không phải tác vụ nào cũng phù hợp để thực thi đồng bộ trong vòng đời của một request HTTP. Các tác vụ tốn thời gian (gửi email, sinh báo cáo, tính toán nặng), tác vụ có lịch chạy định kỳ (đồng bộ dữ liệu hằng đêm, dọn dẹp), hoặc tác vụ cần thực hiện trì hoãn (gửi nhắc nhở sau 24 giờ) — đều cần được tách ra khỏi luồng phản hồi chính. Để giải quyết, hệ thống cần một **cơ chế hàng đợi tác vụ kết hợp bộ lập lịch** với các loại job tiêu biểu:

- **Fire-and-forget:** đẩy vào hàng đợi và chạy sớm nhất có thể.
- **Delayed / Scheduled:** chạy vào một thời điểm xác định trong tương lai.
- **Recurring:** chạy lặp theo lịch (hằng giờ, hằng ngày, hoặc theo biểu thức cron).
- **Continuation:** chạy ngay sau khi một job khác hoàn tất, tạo thành chuỗi nghiệp vụ.

Một yêu cầu quan trọng là **persistence**: trạng thái và hàng đợi job phải được lưu vào kho dữ liệu bền vững, để hệ thống có thể khôi phục lại sau khi khởi động lại hay sự cố — không mất job đang chờ.

Về kiến trúc triển khai, có hai lựa chọn: chạy worker trong cùng tiến trình của dịch vụ nghiệp vụ, hoặc tách worker thành một dịch vụ độc lập. Tách riêng giúp **mở rộng worker theo nhu cầu** mà không ảnh hưởng tới luồng phục vụ request.

2.11. Containerization và Container Orchestration

Container là cơ chế đóng gói ứng dụng cùng toàn bộ môi trường chạy (mã nguồn, thư viện phụ thuộc, biến môi trường, cấu hình) vào một **image bất biến**, có thể vận hành thống nhất trên mọi máy chủ hỗ trợ. So với máy ảo (Virtual Machine), container nhẹ hơn rất nhiều vì chia sẻ kernel của hệ điều hành chủ, đồng thời khởi động nhanh hơn nhiều lần.

Containerization giải quyết bài toán "*chạy được trên máy tôi nhưng không chạy được trên máy khác*" — vốn rất phổ biến khi triển khai hệ thống nhiều dịch vụ với các phụ thuộc khác nhau.

Trong môi trường có nhiều container, cần thêm một lớp công cụ **điều phối container** (container orchestration) để mô tả quan hệ giữa chúng, mạng nội bộ, biến môi trường, dung lượng lưu trữ, và thứ tự khởi động. Ở mức phát triển và demo, các công cụ điều phối đơn máy là đủ. Ở mức production, các nền tảng điều phối cụm (cluster orchestration) bổ sung các năng lực: tự động mở rộng (auto-scaling), cập nhật không gián đoạn (rolling update), khám phá dịch vụ (service discovery), tự khôi phục khi sự cố (self-healing), và cân bằng tải.

2.12. Kubernetes: kiến trúc và các đối tượng cốt lõi

Kubernetes (k8s) là nền tảng điều phối container phổ biến nhất hiện nay, được Google công bố mã nguồn mở năm 2014 và hiện thuộc CNCF. Trong hệ thống microservices, Kubernetes đóng vai trò *hệ điều hành phân tán*: nó tiếp nhận mô tả **trạng thái mong muốn** của ứng dụng (declarative configuration) và liên tục so sánh với **trạng thái thực tế** để điều chỉnh — bù đắp pod chết, mở rộng/thu nhỏ replica, định tuyến lưu lượng, đảm bảo lưu trữ bền vững.

Kiến trúc cụm: Một cluster Kubernetes gồm hai loại node:

- **Control plane (Master):** chứa các thành phần điều phối — kube-apiserver (API duy nhất nhận lệnh từ client), etcd (kho lưu trạng thái nhất quán theo thuật

toán Raft), kube-scheduler (chọn node phù hợp cho pod mới), và kube-controller-manager (chạy các vòng lặp đối chiếu trạng thái).

- **Worker node:** chứa kubelet (đại lý nhận lệnh từ control plane, quản lý vòng đời pod), kube-proxy (định tuyến mạng cấp pod) và một container runtime (containerd, CRI-O).

Người dùng tương tác với cluster qua kubectl (CLI) hoặc các công cụ cao hơn như Helm, Kustomize, ArgoCD; mọi lệnh đều biến đổi thành thao tác CRUD trên API server.

Các đối tượng cốt lõi.

- **Pod:** đơn vị triển khai nhỏ nhất, bao gồm một hoặc vài container chia sẻ chung mạng và volume. Trong thực tế, hầu hết pod chứa một container duy nhất; pattern *sidecar* (logging agent, proxy) thêm container phụ vào cùng pod.
- **Deployment:** quản lý một bộ pod **stateless** có thể thay thế lẫn nhau, hỗ trợ rolling update và rollback. Dùng cho microservice không lưu trạng thái cục bộ.
- **StatefulSet:** tương tự Deployment nhưng cấp **định danh ổn định** (pod-0, pod-1...) và **volume cố định** cho từng pod. Dành cho database hoặc các thành phần stateful (MongoDB, PostgreSQL, RabbitMQ, Elasticsearch).
- **DaemonSet:** đảm bảo mỗi node có chạy đúng một pod (log shipper, node exporter, CNI agent).
- **Job / CronJob:** chạy tác vụ một lần (Job) hoặc theo lịch (CronJob).
- **Service:** cung cấp **địa chỉ ảo cố định** (DNS + ClusterIP) để các pod tìm và gọi nhau bất kể pod thực bên dưới thay đổi. Các loại Service:
 - **ClusterIP** (mặc định): IP nội bộ trong cụm.
 - **NodePort:** mở port trên mọi node để truy cập từ ngoài.
 - **LoadBalancer:** nhờ cloud provider cấp một load balancer ngoài.
 - **Headless** (clusterIP: None): không cấp IP ảo, trả về danh sách pod IP — thường dùng cùng StatefulSet để client tự chọn pod.
- **Ingress:** quy tắc HTTP/HTTPS routing ở biên (host-based, path-based), được thực thi bởi **Ingress Controller** (NGINX, Traefik, HAProxy). Ingress đóng vai

trò *layer-7 reverse proxy* — giúp expose nhiều service ra cùng một địa chỉ bên ngoài.

- **ConfigMap** và **Secret**: tách cấu hình khỏi image. ConfigMap chứa giá trị thường (URL nội bộ, feature flag); Secret chứa giá trị nhạy cảm (mật khẩu, JWT key), được mã hóa base64 và có thể tích hợp KMS.
- **PersistentVolume (PV)** và **PersistentVolumeClaim (PVC)**: trừu tượng hóa lưu trữ. Pod yêu cầu một PVC, Kubernetes liên kết với một PV thực tế (NFS, AWS EBS, hostPath...). Khi pod bị xóa, PV vẫn còn để pod khác attach lại — đây là nền tảng để chạy database trong cluster.
- **HorizontalPodAutoscaler (HPA)**: tự động tăng/giảm số replica của Deployment hoặc StatefulSet theo CPU, memory hoặc custom metric. Yêu cầu cài đặt Metrics Server.
- **Namespace**: cách phân vùng logic resource trong cluster — dùng để cô lập môi trường (dev, staging, prod) hoặc team.
- **NetworkPolicy**: firewall L4 ở cấp pod, quy định pod nào được nói chuyện với pod nào.

Cơ chế self-healing: Mỗi đối tượng quan trọng có một **controller** chạy vòng lặp đối chiếu (reconcile loop): nếu một pod chết, ReplicaSet controller phát hiện sai khác giữa desired = 3 và actual = 2 rồi tạo pod mới; nếu một node mất kết nối, các pod trên đó được lên lịch lại sang node khác. Đây là hiện thân của triết lý **declarative** trong Kubernetes.

Probe: Kubernetes hỗ trợ ba loại probe trên container:

- **livenessProbe**: nếu fail, container sẽ bị restart. Dùng để phát hiện deadlock.
- **readinessProbe**: nếu fail, pod bị tách khỏi Service (không nhận request) — nhưng không restart. Dùng cho khoảng thời gian khởi động chậm hoặc tải nóng cache.
- **startupProbe**: dành cho ứng dụng khởi động lâu, tạm thời "che" hai probe trên cho tới khi pod sẵn sàng.

Lựa chọn cluster runtime cho học tập: Có nhiều bản phân phối nhẹ phù hợp môi trường phòng máy / mạng LAN: **Docker Desktop** (bật tùy chọn Kubernetes — tiện cho 1 máy), **kind** (Kubernetes-in-Docker, chạy node là container, đa node trên cùng 1 máy), **minikube** (VM hoặc Docker), và **k3s** — single-binary, gọn nhẹ, đặc biệt phù hợp khi muốn nối nhiều máy thật trong mạng LAN thành một cluster.

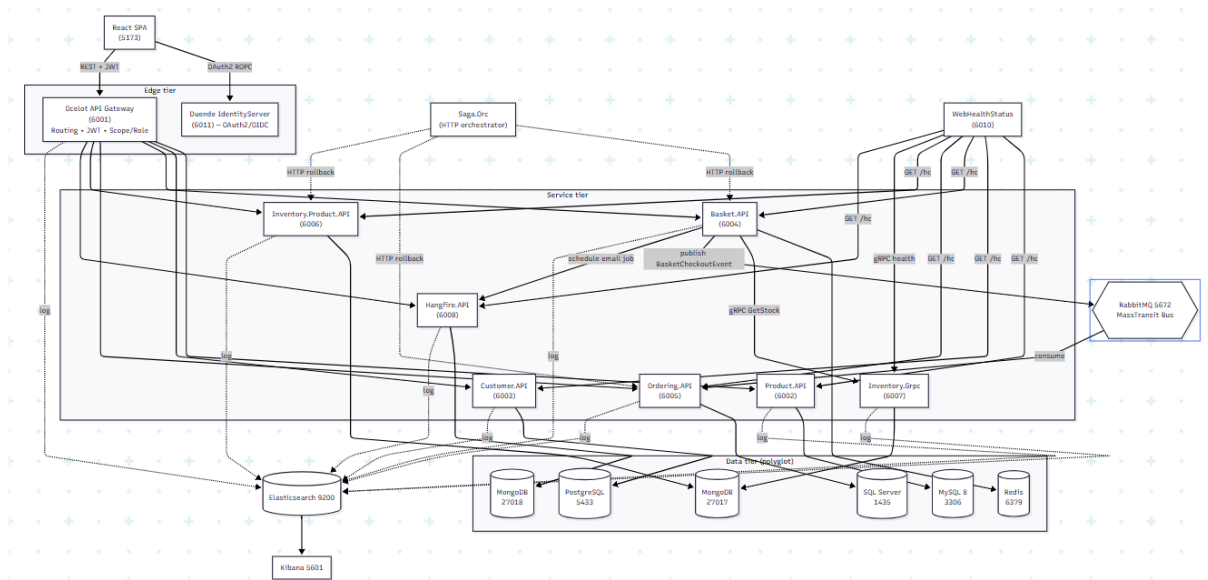
Chương 3. THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng quan

Hệ thống chia làm 4 tầng:

1. **Client tier** — Frontend React SPA, chạy ở trình duyệt; chỉ gọi vào Gateway và IDP.
2. **Edge tier** — Ocelot API Gateway + Identity Provider (Duende IdentityServer).
3. **Service tier** — 7 microservice nghiệp vụ (Product, Customer, Basket, Ordering, Inventory.Product, Inventory.Grpc, Hangfire), 1 saga orchestrator (Saga.Orc).
4. **Infrastructure tier** — 6 database (mỗi loại một service), RabbitMQ, Elasticsearch + Kibana, Hangfire dashboard, Portainer, pgAdmin, WebHealthStatus.

Sơ đồ giao tiếp:



Hình 1: Sơ đồ giao tiếp

3.2. Danh sách microservice và trách nhiệm

Service	Port	DB	Pattern	Vai trò
Product.API	6002	MySQL 8 (EF Core, Pomelo)	Repository	Catalog sản phẩm (No, Name, Price, Description)
Customer.API	6003	PostgreSQL (EF Core, Npgsql)	Repository + Service	Thông tin user (UserName, Email, Address)
Basket.API	6004	Redis (IDistributedCache)	Cache as DB	Giỏ hàng tạm theo user
Ordering.API	6005	SQL Server 2019	DDD + CQRS + MediatR	Đơn hàng (Order Aggregate, OrderCreated domain event)
Inventory.Product.API	6006	MongoDB	Repository	Số kho (purchase / sale entries)
Inventory.Grpc	6007	MongoDB	gRPC service	Cung cấp stock quantity qua gRPC
Hangfire.API	6008	MongoDB	Hangfire	Background jobs (email reminder, welcome jobs)
Saga.Orc	—	(không DB)	State machine	Saga

Service	Port	DB	Pattern	Vai trò
			(Stateless lib)	Orchestration cho luồng Checkout
WebHealthStatus	6010	InMemory	HealthChecks.UI	Dashboard tổng hợp health

3.3. API Gateway (Ocelot)

Tổng quan: Ocelot là API Gateway viết bằng .NET, cấu hình routing bằng file JSON (ocelot.json). Trong dự án dùng 2 file: ocelot.Local.json (dùng trong Docker, downstream qua host.docker.internal) và ocelot.Development.json (chạy trực tiếp localhost).

Tính năng được sử dụng:

- **Routing:** 26+ route map UpstreamPathTemplate → DownstreamPathTemplate + host/port.
- **Authentication:** tích hợp Duende IdentityServer qua AddIdentityServerAuthentication (provider key "Bearer"). Mọi request gửi token JWT trong header Authorization: Bearer
- **Authorization 2 chiều:**
 - AllowedScopes — bắt buộc token có scope tedu_microservices_api.read (cho GET) hoặc tedu_microservices_api.write (cho POST/PUT/DELETE).
 - RouteClaimsRequirement — bắt buộc claim roles=Administrator cho các thao tác quản trị.
- **Polly QoS:** cấu hình QoSOptions cho route /products (TimeoutValue 5000ms, ExceptionsAllowedBeforeBreaking 2, DurationOfBreak 1000ms) — đây là circuit breaker + timeout của Polly.
- **Cache:** Ocelot.Cache.CacheManager (dictionary handle) — sẵn sàng để cache response per-route.
- **Swagger aggregation:** MMLib.SwaggerForOcelot đọc Swagger từ 6 service xuôi (product-service, customer-service, basket-service, order-service, inventory-service, hangfire-service) và gộp thành một UI duy nhất tại /swagger.

- **Token helper:** Gateway có thêm TokenController (GET /api/token) hỗ trợ ROPC từ trong gateway nếu cần.

Phân tầng auth (defense-in-depth):

- Gateway xác minh chữ ký JWT (lớp ngoài) + check scope + check role → block sớm.
- Backend dùng [Authorize] (cho public reads của customer) hoặc [ClaimRequirement(FunctionCode, CommandCode)] (cho admin operations) — fine-grained permission cụ thể (PRODUCT.CREATE, ORDER.UPDATE...).

3.4. Identity Provider (Duende IdentityServer)

Duende IdentityServer là phiên bản thương mại tiếp nối của IdentityServer4, chạy trên .NET 8. Cài đặt:

- **Identity Resources:** OpenID, Profile, Email, custom roles.
- **API Resources:** tedu_microservices_api (audience), gán 2 scope .read và .write.
- **Clients:**
 - tedu_microservices_swagger — Implicit grant, dùng cho UI Swagger của Gateway/Product API.
 - tedu_microservices_postman — Client Credentials + ROPC; chính là client mà frontend dùng.
- **User store:** ASP.NET Identity trên SQL Server (TeduIdentityContext), seed admin alicesmith@example.com / alice123 role Administrator.
- **Permission system:** ngoài role, có entity Permission(Function, Command, RoleId) với khóa kết hợp; lúc cấp token, permission của user được serialize JSON và gán vào claim permissions. Phía service consume claim này qua ClaimRequirementFilter (lập trình lại từ IAuthorizationFilter).

Luồng đăng nhập: Frontend POST connect/token với grant_type=password, username, password, scope=openid profile email roles offline_access tedu_microservices_api.read tedu_microservices_api.write. IDP trả về access_token, refresh_token, expires_in. Frontend lưu vào Zustand store (persist localStorage).

3.5. Các luồng giao tiếp tiêu biểu

(a) Sync REST: Frontend → Gateway → Service

Tất cả thao tác CRUD đều đi qua flow này. Ví dụ tạo đơn:

1. SPA gọi POST `http://localhost:6001/v1/orders` với header `Authorization: Bearer <jwt>`.
2. Gateway match route, check scope `.write`, không yêu cầu role (đơn hàng do customer tự tạo) → forward sang `http://ordering.api/api/v1/orders`.
3. `OrdersController (Ordering.API) [Authorize] → MediatR CreateOrderCommand → handler → EF Core insert SQL Server`.

(b) Sync gRPC: Basket → Inventory.Grpc

Khi user cập nhật giỏ hàng (POST `/baskets`), Basket cần kiểm tra số lượng có sẵn của từng sản phẩm:

1. `BasketsController.UpdateBasket` lặp qua từng `item.ItemNo`, gọi `StockItemGrpcService.GetStock(itemNo)`.
2. Service này gói client gRPC `StockProtoServiceClient` (đã được DI với địa chỉ `http://inventory.grpc`).
3. `Inventory.Grpc` nhận RPC, đọc Mongo, trả về `StockModel { quantity }`.
4. Client gRPC được bọc bằng `Polly AsyncRetryPolicy<StockModel>(3)` để retry khi mạng lỗi.

gRPC ở đây dùng đúng theo lý thuyết: latency thấp, schema chặt, gọi nội bộ (không expose ra ngoài).

(c) Async event: Basket → RabbitMQ → Ordering

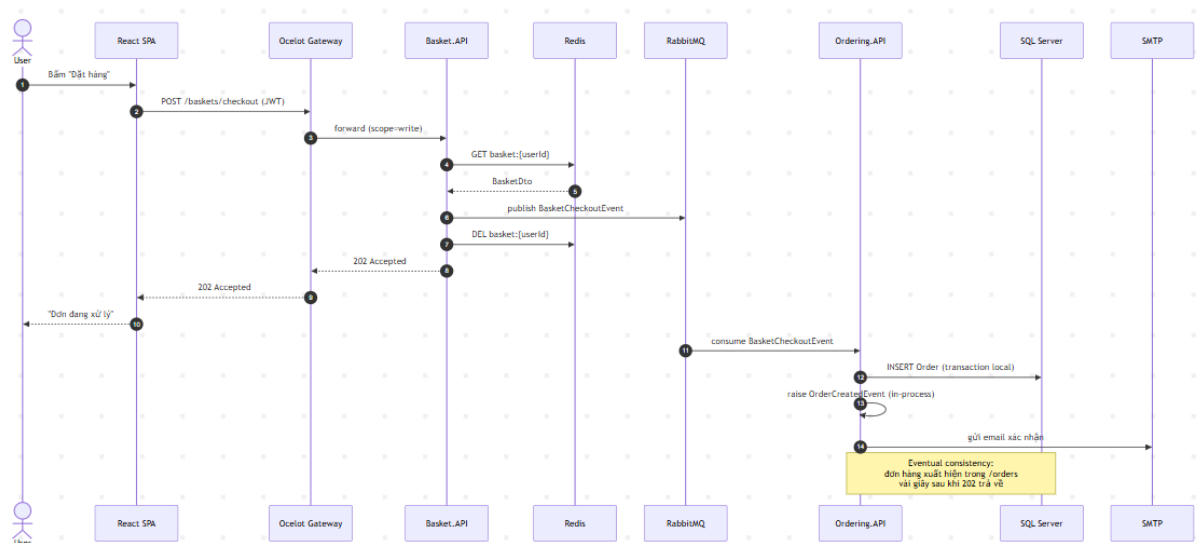
Luồng checkout giỏ hàng:

1. SPA POST `/baskets/checkout`.
2. `BasketsController.Checkout` lấy basket từ Redis, map sang `BasketCheckoutEvent` (gồm `UserName`, `TotalPrice`, `FirstName`, `LastName`, `Email`, `Shipping`, `Invoice`).

3. Gọi `_publishEndpoint.Publish(event)` — MassTransit publish vào exchange (RabbitMQ).
4. Basket xóa cache giỏ hàng (Redis) và trả `Accepted`.
5. `Ordering.API` có `BasketCheckoutEventHandler` : `IConsumer<BasketCheckoutEvent>` lắng nghe queue tương ứng (tên auto theo `KebabCase`).
6. Handler map event sang `CreateOrderCommand`, gọi `_mediator.Send` → tạo Order trong SQL Server → raise domain event `OrderCreatedEvent`.
7. `OrdersDomainHandler` : `INotificationHandler<OrderCreatedEvent>` gửi email xác nhận đơn hàng.

Đây là **choreography saga một bước** (Basket publish, Ordering subscribe), phối hợp loose coupling tự nhiên qua MassTransit.

Sơ đồ tuần tự của luồng checkout async:



Hình 2: Sơ đồ tuần tự luồng checkout async

(d) Saga Orchestration: Saga.Orc

Dự án có thêm một orchestrator riêng (Saga.Orc) demo cách dùng state machine cho saga phức tạp:

- Triển khai bằng thư viện Stateless 5.17.0 — một state machine framework rất nhẹ.
- Các state (EOrderTransactionState): Idle → GetBasket → BasketGot → CreateOrder → OrderCreated → GetOrder → OrderGot → UpdateInventory → InventoryUpdated → DeleteBasket → BasketDeleted.
- Trigger (EOrderAction): GetBasket, CreateOrder, GetOrder, UpdateInventory, DeleteBasket, RollbackOrder.
- Các HttpRepository (BasketHttpRepository, OrderHttpRepository, InventoryHttpRepository) gọi REST sang từng service nội bộ.
- Khi một bước fail (vd UpdateInventory exception), state machine fire RollbackOrder → DeleteInventory entries + DeleteOrder by DocumentNo.

Biểu đồ trạng thái của orchestrator:



Hình 3: Biểu đồ trạng thái của orchestrator

Đây là phương án **dự phòng** cho luồng checkout, áp dụng được khi không muốn dùng event bus, hoặc khi nghiệp vụ cần phải compensate rõ ràng theo từng bước.

(e) Background Job: Email reminder qua Hangfire

- Basket.API có service BackgroundJobHttpService gọi POST /api/schedule-job/send-email lên Hangfire.API với enqueueAt (UTC).
- Hangfire.API serialize job vào MongoDB; khi đến giờ, worker chạy SendEmailContent(email, subject, content) qua SMTP (Gmail SmtpClient).
- HttpClient của Basket có pipeline Polly: UseImmediateHttpRetryPolicy().UseCircuitHttpRetryPolicy().ConfigureTimeoutPolicy() + LoggingDelegatingHandler.

3.6. Mô hình dữ liệu polyglot

Project minh họa rõ polyglot persistence:

- **MySQL** (Product) — quan hệ truyền thống, đọc nhiều, hỗ trợ index tốt cho search.
- **PostgreSQL** (Customer) — quan hệ với ràng buộc unique theo username/email.
- **Redis** (Basket) — ephemeral, key-value, TTL tự nhiên cho giỏ hàng tạm.
- **SQL Server** (Ordering) — transaction phức tạp, hỗ trợ EF Core tốt nhất cho .NET.
- **MongoDB** (Inventory + Hangfire) — schema linh hoạt cho sổ kho nhiều cột phụ và lưu state job dạng tài liệu.

Mỗi service chỉ truy cập DB của chính nó. Khi cần dữ liệu của service khác → gọi qua API (REST/gRPC) hoặc lắng nghe event, không bao giờ query trực tiếp DB khác.

Chương 4. CÀI ĐẶT VÀ TRIỂN KHAI

4.1. Cấu trúc dự án

```

btl_http/
├── Business-Services/src/
│   ├── ApiGateways/OcelotApiGw/      # Ocelot Gateway
│   ├── BuildingBlocks/                # Library dùng chung
│   │   ├── Common.Logging/           # Serilog -> Elasticsearch
│   │   └── Contracts/                 # Interface contract (DateTracking, ApiRe
sult...)
│   └── EventBus.Messages/             # MassTransit message contracts

```

ter	Infrastructure/	# Repository, UoW, Polly policy, Auth fil
	└ Shared/	# Constants, DTOs, Configurations
	Services/	
	└ Product.API/	
	└ Customer.API/	
	└ Basket.API/	
	└ Ordering/	
	└ Ordering.Domain/	# DDD layer
	└ Ordering.Application/	# CQRS + MediatR
	└ Ordering.Infrastructure/	
	└ Ordering.API/	
	└ Inventory/Inventory.Product.API/	
	└ Inventory/Inventory.Grpc/	
	└ Hangfire.API/	
	Saga.Orc/	# Orchestration saga
	WebApps/WebHealthStatus/	# Health UI dashboard
	docker-compose.yml	
	docker-compose.override.yml	
	docker-compose.local-ports.yml	
	tedu-microservices.idp/src/	
	└ TeduMicroservices.IDP/	# Duende IdentityServer
	└ TeduMicroservices.IDP.Infrastructure/	
	└ TeduMicroservices.IDP.Presentation/	
	frontend/	# React SPA
	└ src/	
	└ api/	# axios + module gọi gateway
	└ components/	# UI components + layouts
	└ pages/	# Pages (storefront + admin)
	└ stores/	# Zustand stores
	└ lib/	# JWT decode, format helpers

4.2. Công nghệ sử dụng

Backend (.NET 6 + .NET 8):

Phạm vi	Công nghệ
Web API	ASP.NET Core 6/8
ORM	EF Core 6 (Pomelo MySQL, Npgsql PostgreSQL, SqlServer)
NoSQL	MongoDB.Driver
Cache	Microsoft.Extensions.Caching.StackExchangeRedis
API Gateway	Ocelot 18.0 + Polly + CacheManager + SwaggerForOcelot
Identity	Duende IdentityServer 7 + ASP.NET Identity
Mediator	MediatR 10
Validation	FluentValidation
Mapping	AutoMapper
Message bus	MassTransit 8.0.5 + RabbitMQ
RPC	Grpc.AspNetCore (server) + Grpc.Net.Client (client)
Background job	Hangfire.NetCore + Hangfire.Mongo
Resilience	Polly 7 (Retry, CircuitBreaker, Timeout)
Logging	Serilog + Serilog.Sinks.Elasticsearch
Health Checks	AspNetCore.HealthChecks (MySQL, Redis, Mongo, SqlServer, Npgsql) + UI
Saga	Stateless 5.17 (state machine)

Frontend:

Phạm vi	Công nghệ
UI framework	React 18.3 + Vite 5
Type	TypeScript 5.6
Style	TailwindCSS 3.4
State (client)	Zustand 4.5 (+ persist localStorage)
State (server)	TanStack React Query 5
HTTP	axios 1.7
Routing	react-router-dom 6.27
Form	react-hook-form
Notification	react-hot-toast

4.3. Frontend (React SPA)

Tổ chức routing. Hai layout chính:

- StorefrontLayout — Header + Footer cho khách hàng:
 - / — trang chủ giới thiệu + sản phẩm nổi bật.
 - /products, /products/:id — danh sách và chi tiết sản phẩm.
 - /cart — giỏ hàng (Zustand persist).
 - /checkout — thanh toán (yêu cầu đăng nhập, hỗ trợ Basket checkout qua RabbitMQ hoặc Direct order qua REST).
 - /orders, /orders/:id — lịch sử đơn hàng và chi tiết.
- AdminLayout — Sidebar dark, route /admin/*, bảo vệ bằng ProtectedRoute requireAdmin:
 - Dashboard, ProductsManage, OrdersManage, InventoryManage, CustomersManage, JobsManage.

Auth flow: api/auth.ts gọi trực tiếp IDP (/connect/token) bằng ROPC. Token được Zustand store persist trong localStorage (tedu-auth). 2 axios instance:

- gatewayClient (baseUrl http://localhost:6001) cho mọi API nghiệp vụ.
- identityClient (baseUrl http://localhost:6011) cho các tác vụ liên quan IDP.

Interceptor định Authorization: Bearer ... từ store; response error 401 → logout + redirect.

Defensive UI: Mọi page đều có 3 trạng thái UI: loading (Spinner), error (toast đỏ), empty (EmptyState component). Có ErrorBoundary bao quanh App để bắt lỗi runtime.

4.4. Triển khai bằng Docker Compose

Toàn hệ thống gồm 22 container trong network tedu_microservices (bridge):

- **Database (6):** productdb (MySQL), customerdb (Postgres), basketdb (Redis), orderdb (MSSQL), inventorydb (Mongo), hangfiredb (Mongo).
- **Infrastructure (5):** rabbitmq (5672/15672), elasticsearch (9200), kibana (5601), pgadmin (5050), portainer (8080/9000).
- **Microservices (9):** product.api, customer.api, basket.api, ordering.api, inventory.product.api, inventory.grpc, apigw.ocelot, hangfire.api, webstatus.
- **Identity (2):** tedu.identity.api, tedu_identity_db (MSSQL riêng).

Service-to-service gọi nhau qua DNS container (vd http://inventory.grpc, amqp://guest:guest@rabbitmq:5672, http://tedu.identity.api). apigw.ocelot map sang host.docker.internal:600X ở môi trường Local để demo dễ hơn.

Khởi động:

```
# Backend
cd Business-Services\src
docker compose -p src -f docker-compose.yml -f docker-compose.override.yml -f docker-compose.local-ports.yml up -d --build

# Identity
cd ../../tedu-microservices.idp\src
docker compose up -d --build

# Frontend
cd ../../frontend
npm install
npm run dev
```

4.5. Triển khai trên Kubernetes

Bên cạnh Docker Compose dùng cho phát triển, hệ thống được đóng gói sẵn để chạy trên một cụm Kubernetes đơn-máy thông qua **Docker Desktop Kubernetes** (cluster nội bộ của Docker Desktop trên Windows, thực chất là một VM Linux chứa control plane + node duy nhất). Toàn bộ manifest đặt trong thư mục k8s/.

4.5.1. Cấu trúc manifest

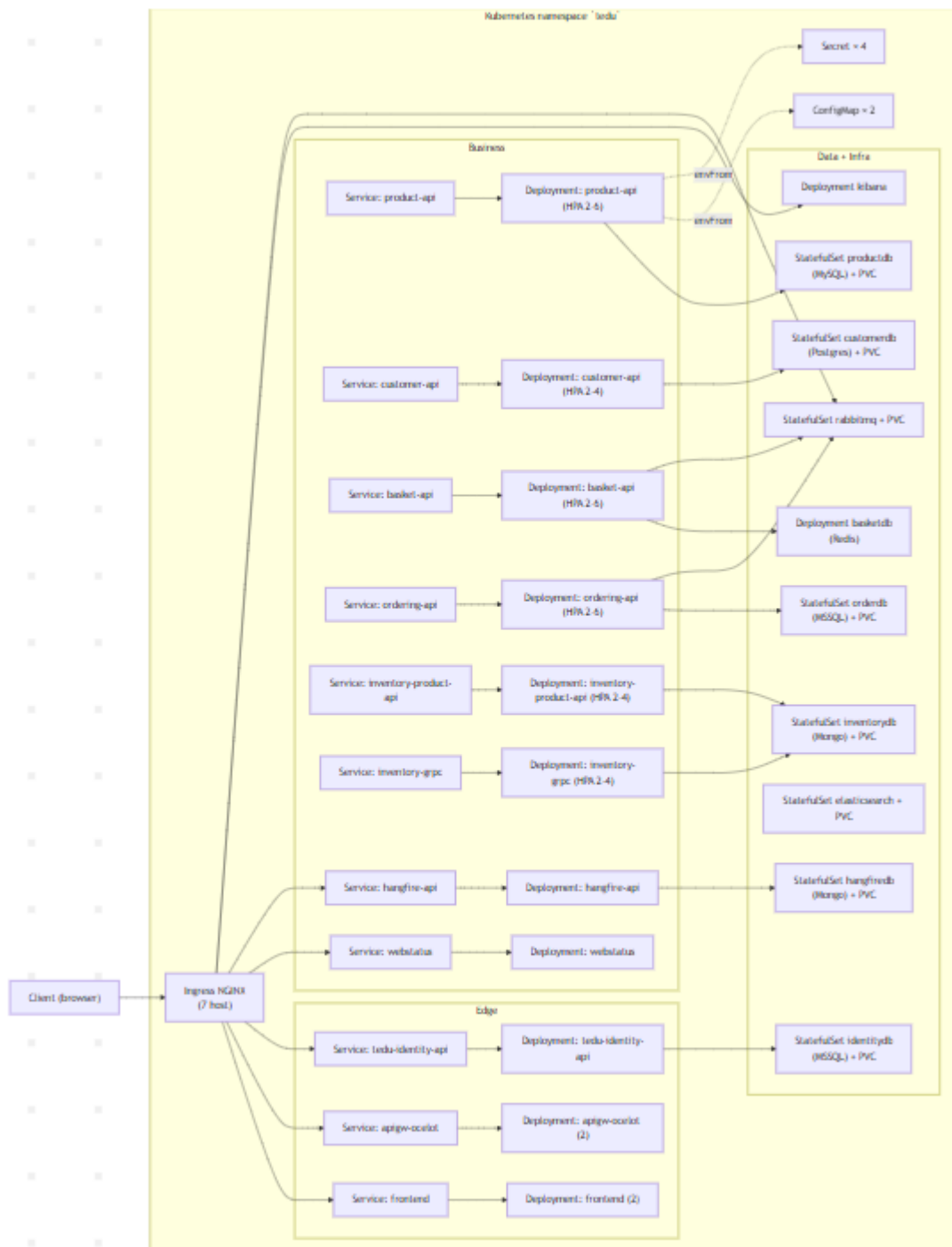
```

k8s/
├── 00-namespace.yaml          # Namespace `tedu`
├── 01-secrets.yaml           # DB / RabbitMQ / Elastic credentials
├── 02-configmaps.yaml        # ConfigMap chung + cấu hình HealthChecks UI
├── 10-databases/              # MySQL, Postgres, Redis, MSSQL x2, MongoDB x2
├── 20-infrastructure/         # RabbitMQ, Elasticsearch, Kibana
├── 30-services/              # 11 service (10 backend + frontend)
├── 40-ingress.yaml           # NGINX Ingress (api, idp, kibana, jobs...)
├── 50-hpa.yaml               # HorizontalPodAutoscaler
├── kustomization.yaml         # Gom toàn bộ resource bằng Kustomize
├── scripts/
│   ├── build-images.ps1      # Build 11 Docker image (tag :k8s)
│   ├── deploy.ps1           # kubectl apply -k + chờ rollout
│   └── teardown.ps1          # Gỡ toàn bộ (giữ hoặc xóa PVC)

```

Tổng cộng sau khi Kustomize render: **57 resource Kubernetes** (1 Namespace, 2 ConfigMap, 4 Secret, 7 StatefulSet, 14 Deployment, 18 Service, 1 Ingress, 7 HPA, 3 PVC template).

Sơ đồ topology các đối tượng Kubernetes:



Hình 4: Sơ đồ topology các đối tượng Kubernetes

4.5.2. Ánh xạ từ Docker Compose sang Kubernetes

Thành phần (Compose)	Kiểu k8s	Tên k8s	Ghi chú
productdb (MySQL)	StatefulSet + headless Service	productdb	PVC 2Gi cho /var/lib/mysql
customerdb (Postgres)	StatefulSet + headless Service	customerdb	PVC 2Gi, PGDATA set rõ
basketdb (Redis)	Deployment + Service	basketdb	Cache, không cần PVC

Thành phần (Compose)	Kiểu k8s	Tên k8s	Ghi chú
orderdb (MSSQL 2019)	StatefulSet + headless Service	orderdb	PVC 3Gi
tedu_identity_db (MSSQL 2022)	StatefulSet + headless Service	identitydb	PVC 3Gi
inventorydb (Mongo)	StatefulSet + headless Service	inventorydb	PVC 2Gi
hangfiredb (Mongo)	StatefulSet + headless Service	hangfiredb	PVC 2Gi, có auth
rabbitmq	StatefulSet + Service (5672/15672)	rabbitmq	PVC 1Gi cho queue durable
elasticsearch	StatefulSet + headless Service	elasticsearch	initContainer chỉnh quyền, PVC 3Gi
kibana	Deployment + Service	kibana	Trỏ tới elasticsearch:9200
product.api → hangfire.api, apigw.ocelot, webstatus	Deployment + Service	đồng tên (kebab-case)	replicas=2 cho gateway và 5 service nghịệp vụ
tedu.identity.api	Deployment + Service	tedu- identity-api	Trỏ DB sang identitydb, env IssuerUri ổn định
Frontend (React + Vite)	Deployment + Service	frontend	Multi-stage Dockerfile build → nginx serve dist/
docker-compose port mapping	Ingress	tedu-ingress	NGINX, host-based: tedu.local, api.tedu.local, idp.tedu.local, kibana.tedu.local, rabbitmq.tedu.local, health.tedu.local, jobs.tedu.local
(mới)	HorizontalPodAutoscaler	7 HPA	CPU 60–75 %, min 2 – max 4 / 6

Để các pod tìm thấy nhau, biến môi trường DatabaseSettings__ConnectionString, GrpcSettings__StockUrl, EventBusSettings__HostAddress... được trỏ tới tên Service (DNS nội bộ dạng <service>.<namespace>.svc.cluster.local, hoặc rút gọn thành <service> trong cùng namespace). Ví dụ:

```
- name: DatabaseSettings__ConnectionString
  value: "Server=productdb;Port=3306;Database=ProductDb;Uid=root;Pwd=$(DB_PASSWORD)"
- name: GrpcSettings__StockUrl
```

```

value: "http://inventory-grpc"
- name: EventBusSettings__HostAddress
  valueFrom:
    configMapKeyRef:
      name: common-config
      key: EventBusSettings__HostAddress # amqp://guest:guest@rabbitmq:5672

```

4.5.3. ConfigMap, Secret và phân tách cấu hình

- common-config (ConfigMap): các biến dùng chung (ASPNETCORE_ENVIRONMENT, ASPNETCORE_URLS, ElasticConfiguration__Uri, ApiConfig__IdentityServerBaseUrl, EventBusSettings__HostAddress). Mọi Deployment đính kèm bằng envFrom: configMapRef.
- healthchecks-ui-config (ConfigMap): danh sách endpoint /hc cho webstatus — y nguyên cấu trúc 7 health check + gRPC health.
- db-credentials (Secret): mssql-sa-password, mysql-root-password, postgres-user, postgres-password, mongo-root-username, mongo-root-password.
- rabbitmq-credentials, elastic-credentials, pgadmin-credentials (Secret).

Connection string lắp ráp tại runtime bằng cú pháp \$(DB_PASSWORD) để Kubernetes substitute giá trị từ Secret — tránh phải hardcode mật khẩu trong manifest.

4.5.4. Health probe và rolling update

Mỗi Deployment ASP.NET đều khai báo:

```

readinessProbe:
  httpGet: { path: /hc, port: 80 }
  initialDelaySeconds: 20
  periodSeconds: 10
livenessProbe:
  httpGet: { path: /hc, port: 80 }
  initialDelaySeconds: 60
  periodSeconds: 30

```

- readinessProbe trở tới endpoint **AspNetCore.HealthChecks** đã tồn tại sẵn (/hc) — pod chỉ nhận traffic sau khi DB, broker, cache đều OK.
- livenessProbe cùng URL nhưng chu kỳ thừa hơn và initialDelaySeconds rộng hơn để tránh restart đột ngột khi service nóng cache.
- inventory-grpc (HTTP/2) dùng tcpSocket probe vì /hc không phù hợp với client gRPC.

- Cluster mặc định dùng chiến lược RollingUpdate: pod mới được scale lên trước, qua probe rồi mới gỡ pod cũ → đảm bảo zero-downtime.

4.5.5. Ingress và truy cập từ ngoài

Tất cả lưu lượng đi vào cluster đi qua **NGINX Ingress Controller** với 7 host:

Host	Service đích	Vai trò
tedu.local	frontend	SPA React
api.tedu.local	apigw-ocelot	API Gateway
idp.tedu.local	tedu-identity-api	Duende IdentityServer
kibana.tedu.local	kibana	Centralized log UI
rabbitmq.tedu.local	rabbitmq (15672)	RabbitMQ management UI
health.tedu.local	webstatus	HealthChecks UI
jobs.tedu.local	hangfire-api	Hangfire dashboard

Ingress bật CORS cho tedu.local, đặt proxy-body-size: 10m cho upload và dùng ingressClassName: nginx (chuẩn API mới thay cho annotation cũ).

4.5.6. Horizontal Pod Autoscaler

50-hpa.yaml cấu hình HPA cho 7 đối tượng (product-api, customer-api, basket-api, ordering-api, inventory-product-api, inventory-grpc, apigw-ocelot). Mỗi HPA bám theo CPU (60–75 %) và memory (chỉ riêng product-api), minReplicas=2, maxReplicas=4–6. Khi tải tăng (giả lập bằng kubectl run loadtest), Kubernetes sẽ tự scale tăng pod sau khi quan sát metric trong vòng 1–2 phút.

Lưu ý vận hành: HPA chỉ hoạt động khi đã cài Metrics Server trong cluster. Nếu không, các Deployment vẫn chạy ở mức minReplicas.

4.5.7. Quy trình triển khai

```
# 1) Build 11 image (.NET API + React + nginx) tag :k8s
.\k8s\scripts\build-images.ps1

# Nếu dùng kind:
.\k8s\scripts\build-images.ps1 -KindLoad -KindClusterName kind

# Nếu dùng minikube:
.\k8s\scripts\build-images.ps1 -MinikubeLoad

# 2) Apply toàn bộ Kustomize
.\k8s\scripts\deploy.ps1

# 3) Map host (Windows, dòng C:\Windows\System32\drivers\etc\hosts)
# 127.0.0.1 tedu.local api.tedu.local idp.tedu.local kibana.tedu.local rabbitmq.tedu.local health.tedu.local jobs.tedu.local

# 4) Gỡ
```

```
.\k8s\scripts\teardown.ps1 # mặc định xóa PVC  
.\k8s\scripts\teardown.ps1 -KeepData # giữ PVC, gỡ workload
```

Script `deploy.ps1` còn tự chờ rollout status của mọi `StatefulSet` + `Deployment` để biết hệ thống đã thực sự sẵn sàng.

4.5.8. Lựa chọn thiết kế và đánh đổi

- **StatefulSet cho database:** cấp tên ổn định (`productdb-0`, `inventorydb-0`) — quan trọng vì connection string cố định tham chiếu hostname.
- **Headless Service** (`clusterIP: None`) trước `StatefulSet` để DNS trả thẳng IP pod, phù hợp với database 1 instance + tránh proxy không cần thiết.
- **PVC tách rời pod:** dữ liệu sống sót khi pod restart, đúng tinh thần *cattle*, *not pets* nhưng vẫn lưu state.
- **imagePullPolicy: IfNotPresent** + tag `:k8s`: phù hợp cho local cluster, không cần registry. Trong production cần đẩy lên registry (Harbor, Docker Hub, GHCR) và đổi sang tag SHA + Always.
- **Ingress thay cho NodePort:** chỉ một entry point HTTP duy nhất, dễ bật TLS sau này bằng cert-manager.
- **Polyglot DB vẫn được tôn trọng:** mỗi service một `StatefulSet/Deployment` riêng — không gom database vào một pod chung như anti-pattern thường gặp.

4.5.9. Phạm vi và hạn chế

Phạm vi triển khai k8s của đề tài **dừng ở mức cluster đơn-máy trên Docker Desktop Kubernetes** — đủ để minh họa các pattern của Kubernetes (declarative config, scheduling, scale-out, self-healing, service discovery, rolling update) trong khuôn khổ một bài tập lớn. Hạn chế trực tiếp đi kèm phạm vi này:

- **Một node duy nhất:** Docker Desktop chạy một VM Linux nội bộ làm cluster đơn-node, do đó toàn bộ pod đều schedule trên cùng máy host. HPA tăng/giảm replica nhưng pod không phân bố qua nhiều máy vật lý.
- **Storage hostpath:** PVC dùng StorageClass mặc định của Docker Desktop, dữ liệu nằm trong VM. Khi xóa VM/cluster sẽ mất nếu không backup.

- **Image build cục bộ** với tag :k8s (imagePullPolicy: IfNotPresent) — không cần registry. Khi rebuild image phải kubectl rollout restart để pod pick up image mới.
- **HPA dùng CPU/Memory**: chưa dùng custom metric (RPS, queue depth) — đủ cho mục tiêu minh họa auto-scaling.
- **Saga.Orc chưa triển khai k8s** (chỉ chạy ngoài cluster, gọi qua port-forward) vì là module tùy chọn — có thể bổ sung Deployment riêng tương tự.
- **Yêu cầu tài nguyên cho host**: Docker Desktop mặc định cấp 2 CPU + 2 GB RAM cho VM Linux — không đủ. Cần vào Settings → Resources kéo lên tối thiểu **4 CPU + 8 GB RAM** để 22 container + pod scale theo HPA chạy mượt.

4.5.10. Hướng dẫn mở rộng cluster sang nhiều máy trong mạng LAN

*Lưu ý phạm vi. Toàn bộ code và manifest đi kèm đề tài chỉ triển khai cluster đơn-máy trong Docker Desktop vì phòng máy hiện không có Linux server. Phần dưới đây trình bày **quy trình lý thuyết** để khi có điều kiện (Linux host hoặc Linux VM), bộ manifest k8s/ có thể tái sử dụng cho cụm nhiều máy mà chỉ cần bổ sung **bốn khối hạ tầng** quanh nó.*

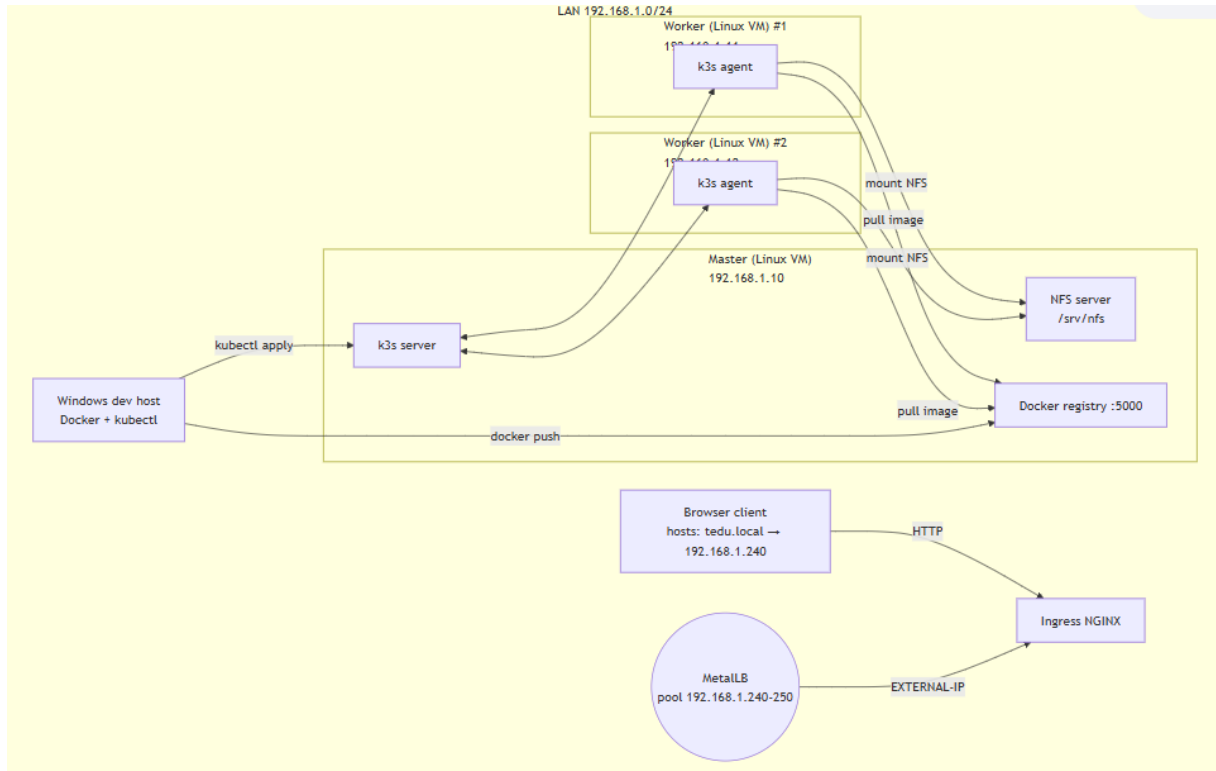
a) Vì sao phải có Linux trên mỗi máy node

Kubernetes server (kube-apiserver, kubelet, etcd...) là phần mềm Linux-only — không có bản native cho Windows. Windows có thể làm worker chạy *Windows container*, nhưng vẫn phải join vào một control plane Linux. Vì vậy mỗi máy node trong cluster đa-máy đều cần một **runtime Linux**:

- **Multipass** (Canonical): cài qua winget install Canonical.Multipass, mỗi lệnh multipass launch --name node1 --cpus 2 --memory 4G --disk 30G --bridged Ethernet tạo một VM Ubuntu nhẹ có IP từ LAN. Chạy được trên Windows Home/Pro/Edu.
- **WSL2 mirrored mode** (Windows 11 ≥ 22H2): bật networkingMode=mirrored trong ~/.wslconfig để WSL2 chia chung network với host — IP của WSL2 bằng IP của Windows.
- **Hyper-V VM** (Windows Pro/Edu): tạo VM Ubuntu với *External Virtual Switch* (bridge thật vào card mạng).

Sau khi có Linux trong mỗi máy, các bước cài đặt tiếp theo giống hệt cluster Linux bare-metal.

b) Sơ đồ topology mục tiêu



Hình 5: Sơ đồ topology mục tiêu

c) Bốn khối hạ tầng phải bổ sung

(1) Phân phối cluster — k3s đóng gói toàn bộ Kubernetes vào một binary, phù hợp nhất cho mạng LAN học tập. Trên Linux master (vd 192.168.1.10):

```
curl -sfL https://get.k3s.io | sh -s - - server --disable=traefik
sudo cat /var/lib/rancher/k3s/server/node-token # lấy join token
```

Trên mỗi Linux worker:

```
curl -sfL https://get.k3s.io | \
  K3S_URL=https://192.168.1.10:6443 \
  K3S_TOKEN=<token> \
  sh -s - agent
```

Sau đó kubectl get nodes liệt kê đầy đủ các node trạng thái Ready. k3s mặc định dùng **Flannel** (CNI overlay VXLAN qua port 8472/UDP) lo việc mạng pod-to-pod xuyên node — chỉ cần whitelist port trong LAN.

(2) Image registry cục bộ: Manifest dùng tag :k8s với imagePullPolicy: IfNotPresent, nên các node phải pull được image từ một nguồn chung. Chạy Docker registry trên master:

```
docker run -d -p 5000:5000 --restart=always --name registry registry:2
```

Trên mọi node, thêm /etc/rancher/k3s/registries.yaml để containerd tin tưởng registry HTTP:

```
mirrors:
  "192.168.1.10:5000":
    endpoint: ["http://192.168.1.10:5000"]
configs:
  "192.168.1.10:5000":
    tls: { insecure_skip_verify: true }
```

Restart k3s (master) và k3s-agent (worker), rồi từ máy dev docker tag ... 192.168.1.10:5000/<name>:k8s + docker push.

Để không phải sửa tay 11 trường image: trong manifest, thêm vào kustomization.yaml (hoặc tạo overlay):

```
images:
  - { name: product-api,          newName: 192.168.1.10:5000/product-api,      n
    ewTag: k8s }
  - { name: customer-api,        newName: 192.168.1.10:5000/customer-api,    n
    ewTag: k8s }
  # ... 9 image còn lại tương tự
```

Kustomize sẽ tự viết lại toàn bộ image: khi kubectl apply -k.

(3) Shared storage — NFS: Khi PVC dùng StorageClass local-path mặc định, dữ liệu bị giữ trên node hiện tại; nếu pod bị reschedule sang node khác, dữ liệu không đi theo. Lời giải nhẹ nhất cho LAN: dựng NFS server trên một máy và dùng nfs-subdir-external-provisioner.

Trên máy NFS (thường là master):

```
sudo apt install -y nfs-kernel-server
echo "/srv/nfs 192.168.1.0/24(rw,sync,no_subtree_check,no_root_squash)" \
| sudo tee -a /etc/exports
sudo exportfs -ra && sudo systemctl enable --now nfs-server
```

Trong cluster:

```
helm install nfs-prov \
  nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
  -n kube-system \
  --set nfs.server=192.168.1.10 \
  --set nfs.path=/srv/nfs \
  --set storageClass.name=nfs-client \
  --set storageClass.defaultClass=true
```

Sau đó áp một patch Kustomize bơm storageClassName: nfs-client vào tất cả volumeClaimTemplates của StatefulSet trong base:

```
# patches/storage-class.yaml – Kustomize JSON patch
- op: add
  path: /spec/volumeClaimTemplates/0/spec/storageClassName
  value: nfs-client
```

PV mới sẽ là thư mục con trong /srv/nfs, pod đặt ở node nào cũng truy cập được.

(4) IP ảo cho ingress — MetalLB: Service LoadBalancer không có cloud cấp IP trên bare metal. MetalLB lấy một dải IP trong LAN dành riêng cho cluster (vd 192.168.1.240–250) và "phát" qua giao thức ARP/L2 cho ingress controller:

```
# k8s/metallb-pool.yaml
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata: { name: lan-pool, namespace: metallb-system }
spec: { addresses: ["192.168.1.240-192.168.1.250"] }
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata: { name: lan-l2, namespace: metallb-system }
spec: { ipAddressPools: [lan-pool] }
```

Sau khi cài MetalLB controller (kubectl apply -f <https://raw.githubusercontent.com/metallb/metallb/v0.14.8/config/manifests/metallb-native.yaml>) và NGINX Ingress Controller, ingress sẽ nhận một IP từ pool, cập nhật vào file hosts của các máy client trong LAN:

```
192.168.1.240   tedu.local api.tedu.local idp.tedu.local kibana.tedu.local
```

d) Kiểm chứng tính phân tán

Sau khi cluster đa-node sẵn sàng, các tình huống sau minh họa rõ "đa máy" so với "Docker Desktop một máy":

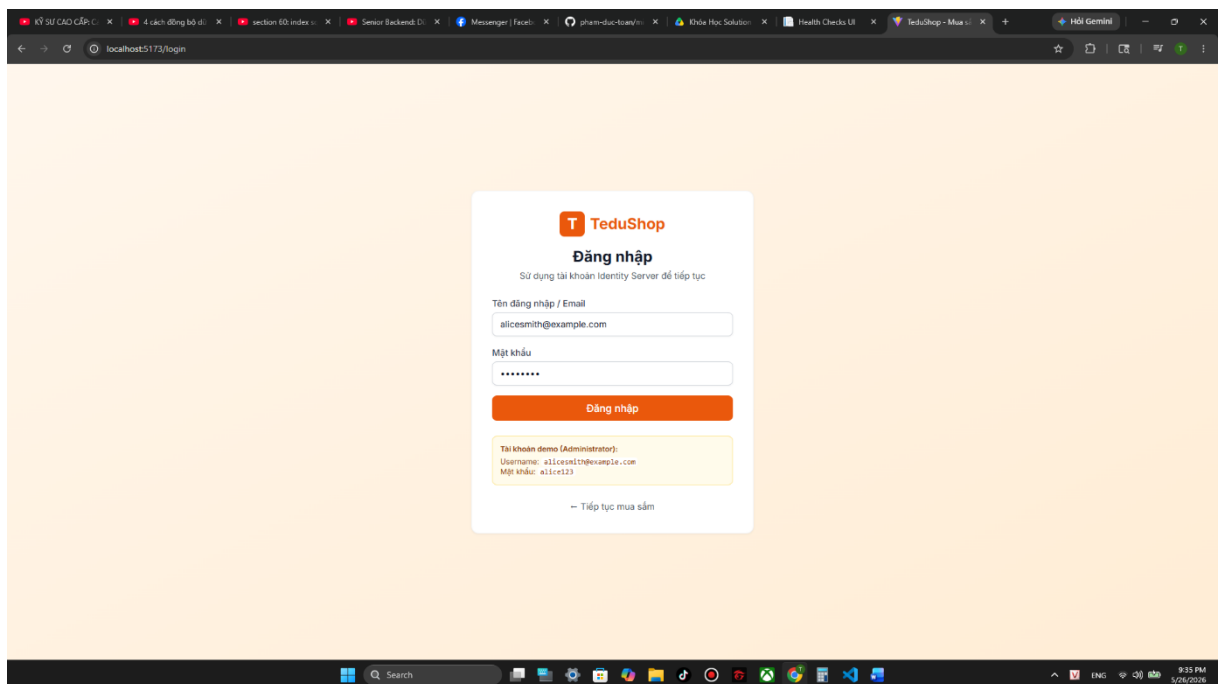
Tình huống	Lệnh	Quan sát kỳ vọng
Pod phân bố qua nhiều node	kubectl -n tedu get pods -o wide	Cột NODE đa dạng — scheduler tự rải workload theo CPU/Memory available
Self-healing cross-node	kubectl -n tedu delete pod -l app=product-api	Pod mới có thể nảy sinh trên <i>worker khác</i> , traffic không gián đoạn nhờ Service ảo

Tình huống	Lệnh	Quan sát kỳ vọng
Scale ngang dưới tải	<code>kubectrl run loadgen --image=busybox -- /bin/sh -c "while true; do wget -q -O- http://api.tedu.local/products; done" rồi kubectrl -n tedu get hpa -w</code>	Replica product-api tăng từ 2 → 4–6 trong 1–2 phút
Tắt một worker	<code>sudo systemctl stop k3s-agent</code> trên 1 máy	Pod trên node đó chuyển trạng thái Terminating, sau pod-eviction-timeout (~5 phút) tái lập lịch sang node còn sống

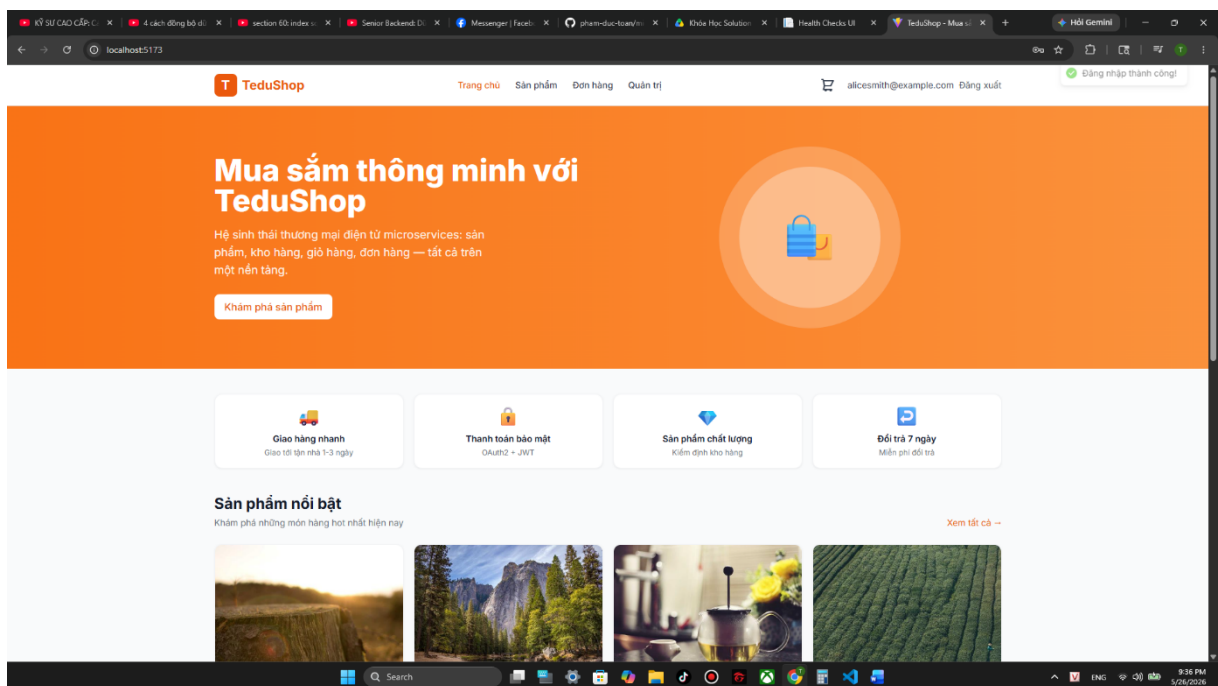
e) Hạn chế còn lại trong phạm vi LAN

- **NFS server là điểm phụ thuộc duy nhất:** nếu máy NFS sập, mọi database mất kết nối storage. Để khắc phục cần Longhorn / Ceph (replicated block storage) — nằm ngoài phạm vi đề tài.
- **Database vẫn 1 instance** (StatefulSet replicas=1). Để DB HA thật cần cluster nội của từng engine (MongoDB ReplicaSet, Postgres Patroni...) — không khả thi trong khuôn khổ học tập.
- **Cluster nội bộ HTTP:** chấp nhận được cho LAN tin cậy, không phơi ra Internet.

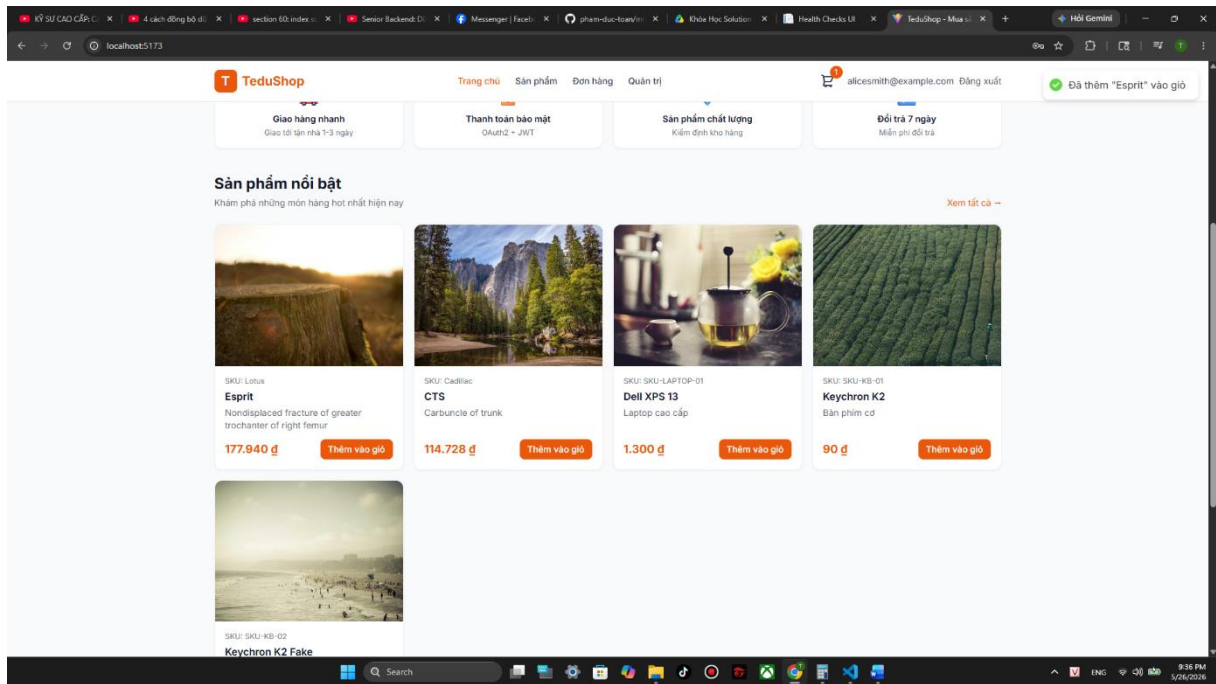
Chương 5. KẾT QUẢ MINH HỌA



Trang đăng nhập

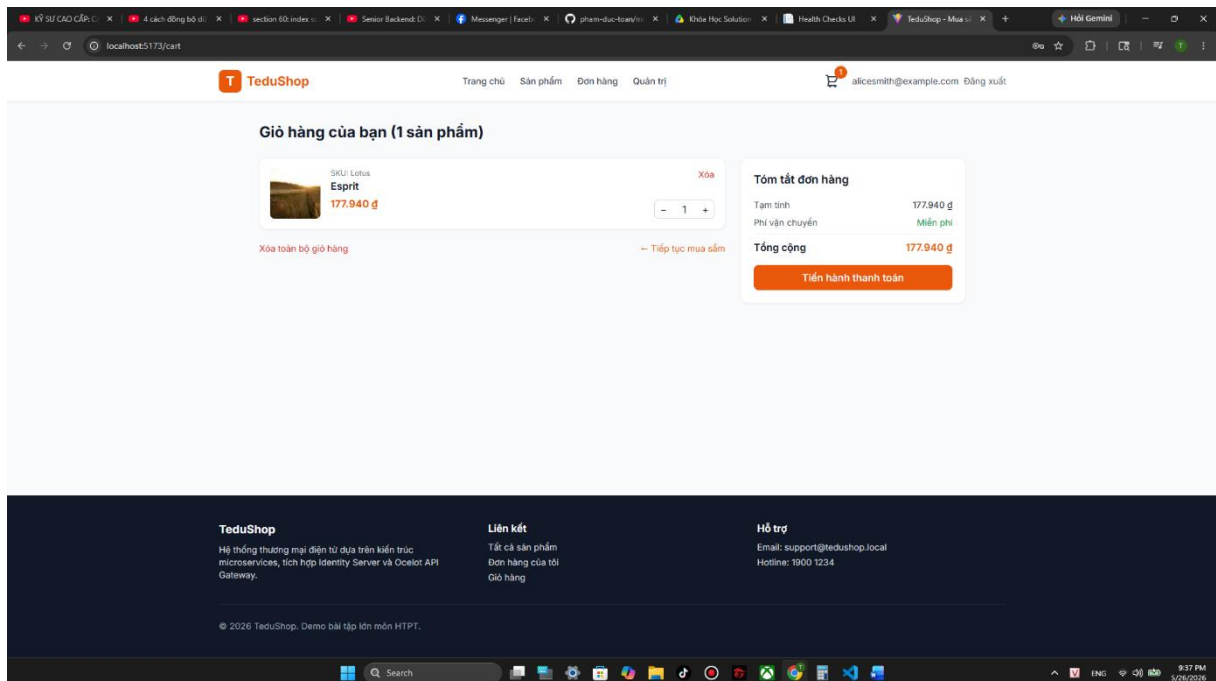


User mở `http://localhost:5173/login`, nhập `alicesmith@example.com` / `alice123`
→ SPA `POST /connect/token` → IDP trả JWT chứa `roles=Administrator` + `claim permissions=[...]` + `scope read/write` → SPA persist token → redirect home.



Trang chủ. GET /products qua gateway → Ocelot check scope .read → forward product.api → MySQL trả danh sách → SPA hiển thị grid.

User add to cart → SPA POST /baskets → Basket.API gọi gRPC Inventory.Grpc.GetStock để biết số lượng có sẵn → set vào CartItem → lưu Redis.



Trang giỏ hàng

Thanh toán

Thông tin khách hàng

Họ: Tên:

Email:

Địa chỉ giao hàng:

Số nhà, tên đường, phường, quận, tỉnh/thành:

Địa chỉ xuất hóa đơn (tùy chọn):

Đơn hàng của bạn

Esprit x 1 177.940 đ

Tạm tính 177.940 đ

Vận chuyển Miễn phí

Tổng 177.940 đ

Người mua: alicsmith@example.com (qua Basket)

Đặt hàng

Phương thức thanh toán

☒ Thanh toán khi nhận hàng (COD)
Thanh toán bằng tiền mặt khi giao hàng

☐ Chuyển khoản ngân hàng
Thanh toán qua tài khoản ngân hàng

Phương thức đặt hàng

☒ Sử dụng quy trình Basket Checkout
Đẩy giỏ hàng lên Basket Service rồi gọi /baskets/checkout (đi qua Inventory gRPC để kiểm tra tồn kho và publish event). Bỏ chọn để gọi thẳng Order API tạo đơn cùng.

User checkout → SPA POST /baskets/checkout → Basket map BasketCheckoutEvent → publish RabbitMQ → trả 202 Accepted → Ordering consume → tạo Order trong MSSQL → raise OrderCreatedEvent (in-process) → send email confirm.

Đơn hàng của tôi

Mã đơn: c9e18398-fc67-4f3b-834f-9b8262f288e3

Toán Phạm — alicsmith@example.com

Giao tới: Dong Anh

1.300 đ

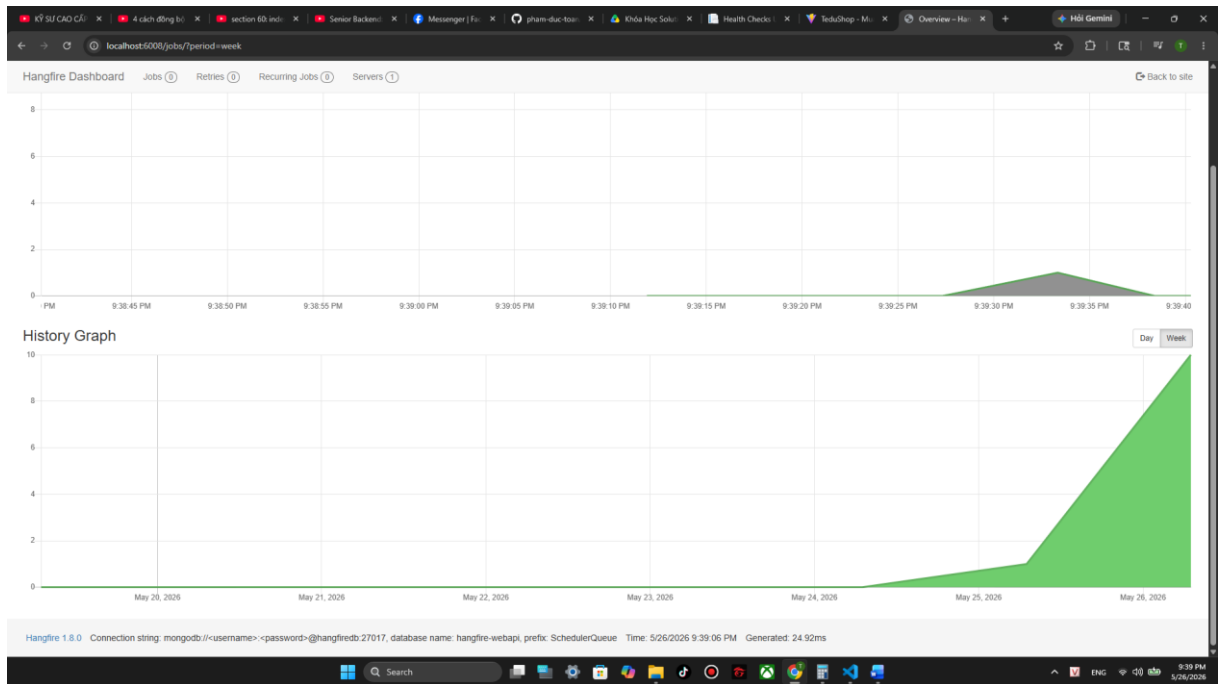
Đặt hàng thành công qua Basket Checkout!

TeduShop
Hệ thống thương mại điện tử dựa trên kiến trúc microservices, tích hợp Identity Server và Ocelot API Gateway.

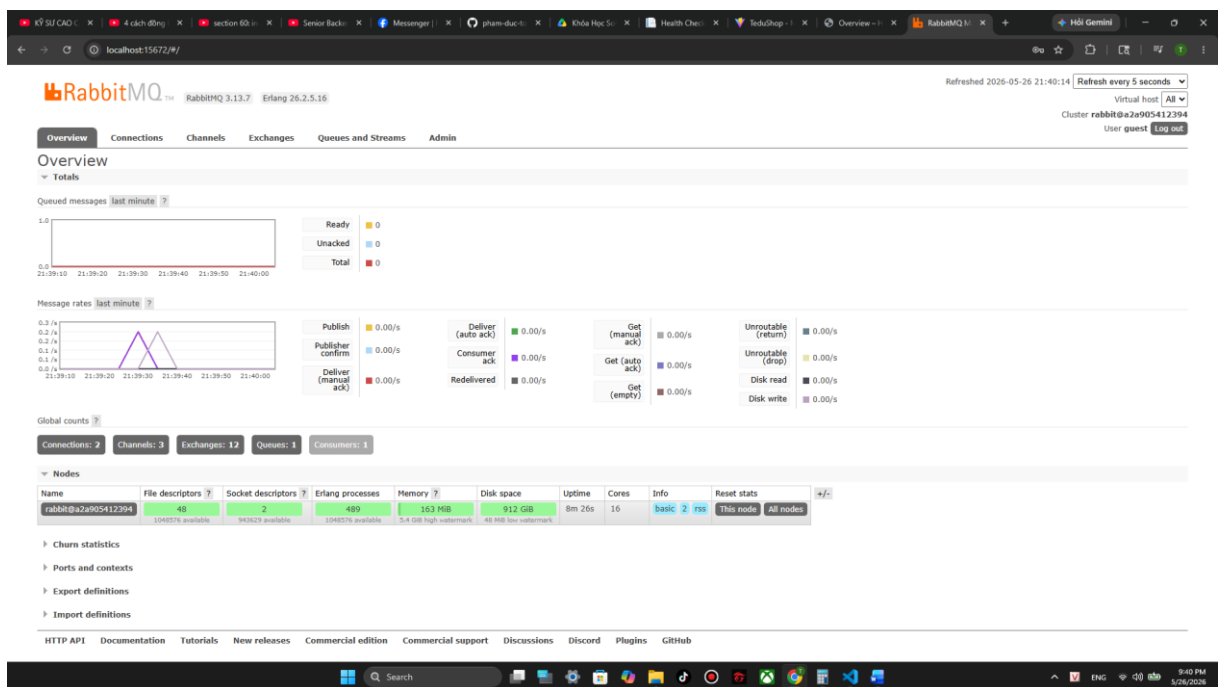
Liên kết
Tất cả sản phẩm
Đơn hàng của tôi
Giỏ hàng

Hỗ trợ
Email: support@tedushop.local
Hotline: 1900 1234

© 2026 TeduShop. Demo bài tập lớn môn HTPPT.



Quản lý hangfire job đã thêm 1 job send mail



Rabbitmq thêm 1 message vào queue

Health Checks Status

Polling interval: 10 secs [Stop polling](#)

	NAME	HEALTH	ON STATE FROM	LAST EXECUTION
+	Product Health Check	OK	2026-05-26T14:33:46.7082311+00:00	21:40:28 26/5/2026
+	Customer Health Check	OK	2026-05-26T14:32:04.6849526+00:00	21:40:28 26/5/2026
+	Basket Health Check	OK	2026-05-26T14:32:25.3945977+00:00	21:40:28 26/5/2026
+	Ordering Health Check	OK	2026-05-26T14:32:35.6890804+00:00	21:40:28 26/5/2026
+	Inventory Health Check	OK	2026-05-26T14:31:53.1679607+00:00	21:40:28 26/5/2026
+	Scheduled Job Health Check	OK	2026-05-26T14:31:53.9136602+00:00	21:40:28 26/5/2026
+	Inventory Grpc (via WebHealthStatus)	OK	2026-05-26T14:31:54.2039496+00:00	21:40:28 26/5/2026

Trang quản lý service health của tất cả service

TeduShop Admin

Khu vực quản trị

Xin chào, alicsmith@example.com

Tổng sản phẩm
5
Giá trung bình: 58.830 g

Đơn hàng của tôi
7
Theo username admin hiện tại

Doanh thu (đơn của tôi)
184.530 g
Cộng dồn TotalPrice

Vai trò hiện tại
Administrator
Quyền: tất cả modules

Sản phẩm mới nhất [Quản lý --](#)

Keychron K2 Fake SKU: SKU-KB-02	90 g
Keychron K2 SKU: SKU-KB-01	90 g
Dell XPS 13 SKU: SKU-LAPTOP-01	1.300 g
CTS SKU: Cadillac	114.728 g
Esprit SKU: Lotus	177.940 g

Đơn hàng gần đây [Quản lý --](#)

0ae559ab-0b05-48c1-3bde-e2f83e772019 Toàn Phạm	Mới 1.300 g
5-b77216-6a09-4031-9f42-d81d3287396c Toàn Phạm	Mới 1.300 g
905b738c-3978-467c-880d-44cbe0b609c2 Toàn Phạm	Mới 90 g
2ae0514e-5768-479c-885c-bc095ea12ef Toàn Phạm	Mới 1.300 g
9e5b4899-21a7-6c43-aa09-78587464a811 Toàn Phạm	Mới 1.300 g

Ngoài ra vẫn đầy đủ CRUD cho các service

T

TeduShop Admin

Dashboard

Sản phẩm

Đơn hàng

Kho hàng

Khách hàng

Tác vụ nền

Về trang khách

Đăng xuất

Khu vực quản trị

Xin chào, alicsmith@example.com

Quản lý sản phẩm

Thêm sản phẩm

Tìm kiếm theo tên, SKU...

ID	SKU	TÊN SẢN PHẨM	TÓM TẮT	GIÁ	THAO TÁC
#1	Lotus	Esprit	Nondisplaced fracture of greater trochanter of right femur	177.940 đ	Sửa Xóa
#2	cad111ac	CTS	Carbuncle of trunk	114.728 đ	Sửa Xóa
#5	SKU-LAPTOP-01	Dell XPS 13	Laptop cao cấp	1.300 đ	Sửa Xóa
#6	SKU-KB-01	Keychron K2	Bàn phím cơ	90 đ	Sửa Xóa
#7	SKU-KB-02	Keychron K2 Fake	Bàn phím cơ	90 đ	Sửa Xóa

T

TeduShop Admin

Dashboard

Sản phẩm

Đơn hàng

Kho hàng

Khách hàng

Tác vụ nền

Về trang khách

Đăng xuất

Khu vực quản trị

Xin chào, alicsmith@example.com

Quản lý đơn hàng

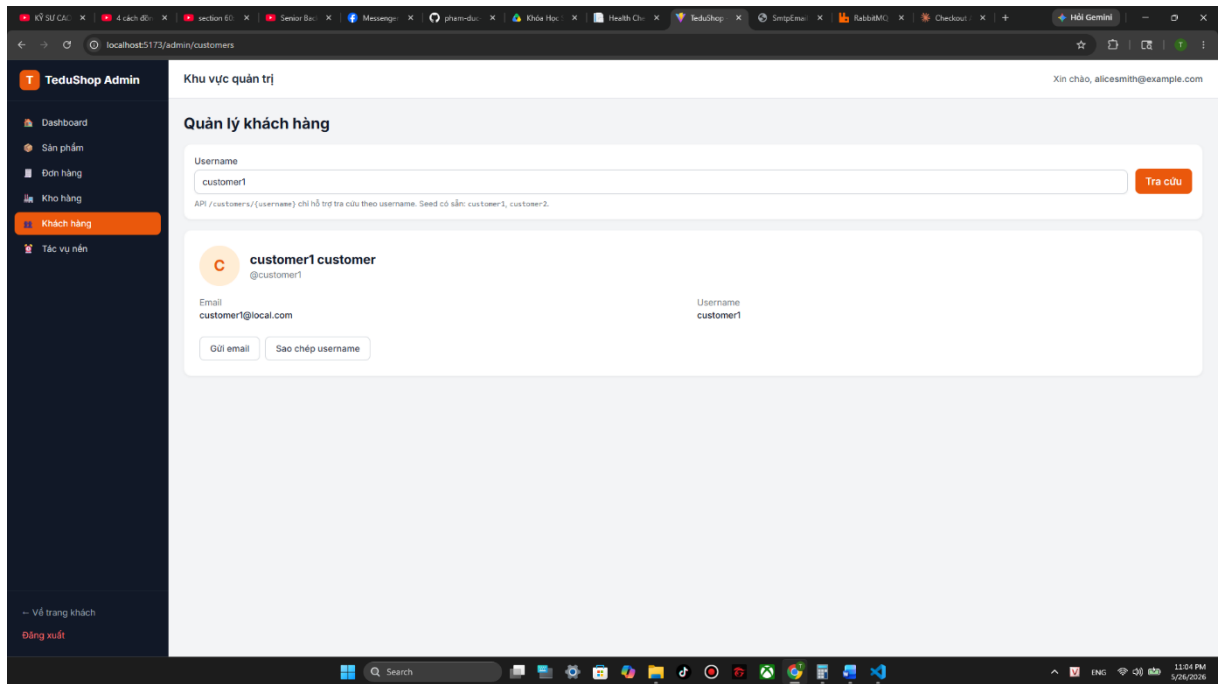
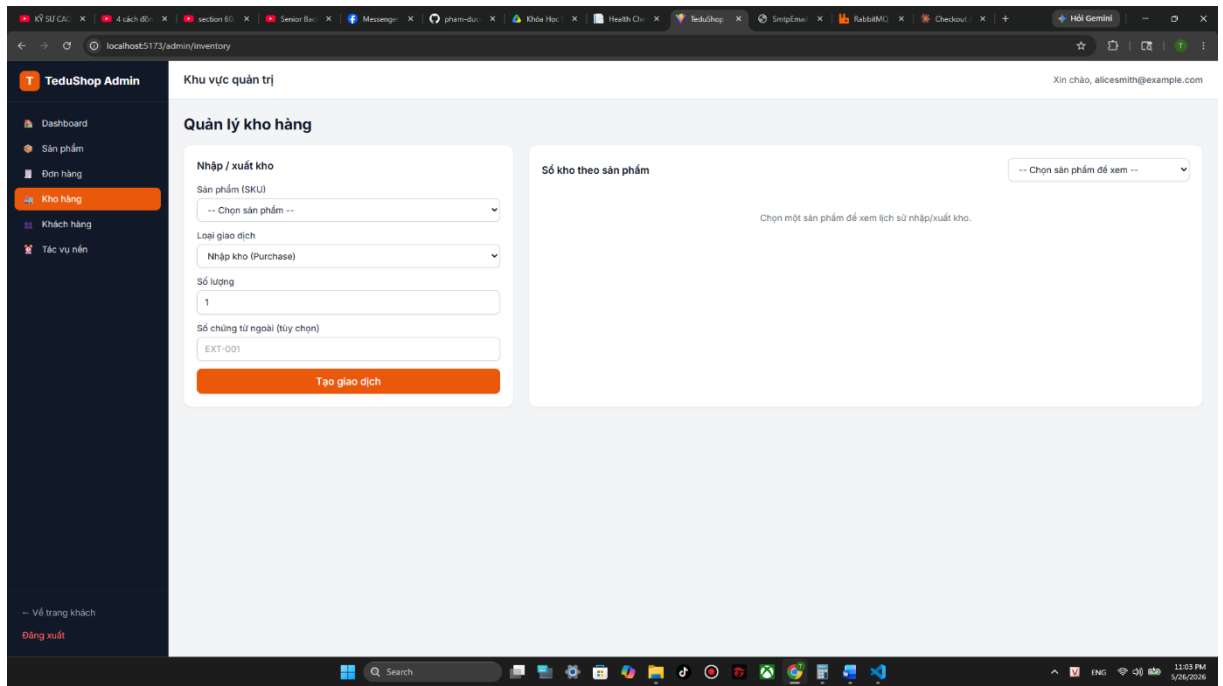
Lọc theo username

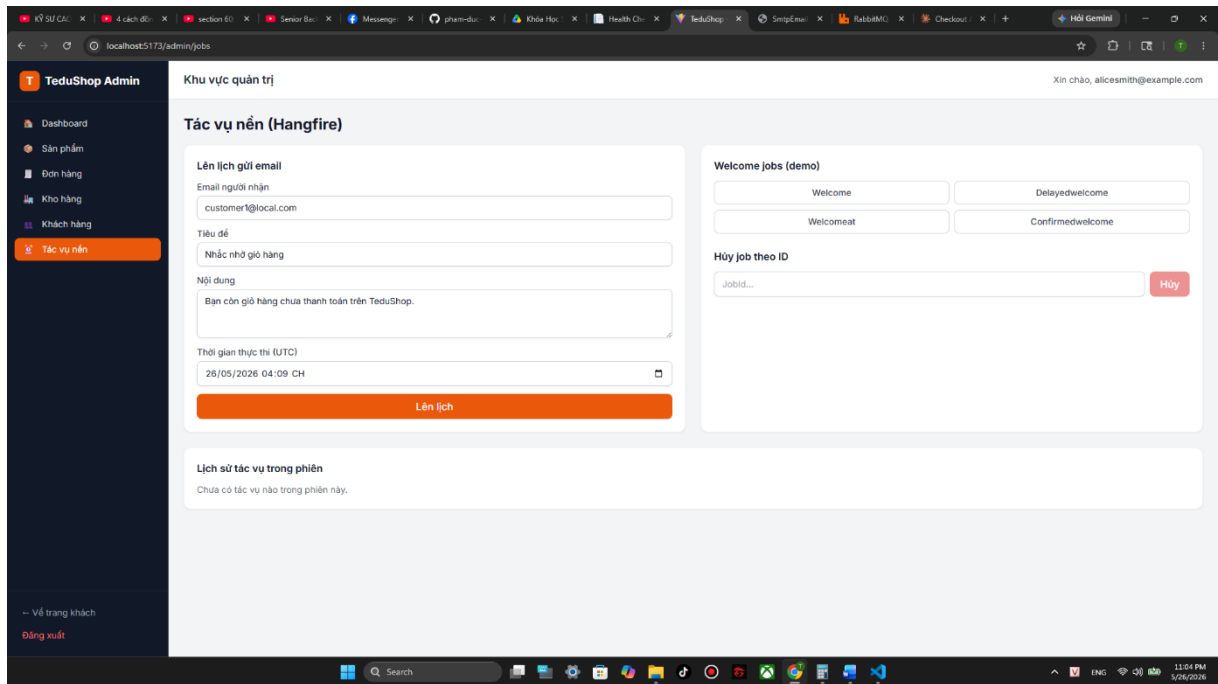
customer1

Tim

API yêu cầu username (không hỗ trợ lọc kế toán bộ).

ID	DOCUMENT NO	KHÁCH HÀNG	EMAIL	ĐỊA CHỈ GIAO	TỔNG TIỀN	TRẠNG THÁI	THAO TÁC
#1	fa7e5196-3b31-4b72-8e4d-cb9e56b5eFB1	customer1 customer	customer1@local.com	Wollongong	250 đ	Mới	Xem Sửa Xóa





Chương 6. ĐÁNH GIÁ VÀ KẾT LUẬN

6.1. Những gì đã đạt được

- 9+ microservice nghiệp vụ với **database-per-service polyglot** (MySQL, PostgreSQL, Redis, SQL Server, MongoDB).
- **3 mô hình giao tiếp** đặc trưng: REST sync qua Gateway, gRPC nội bộ (Basket → Inventory), Event async qua RabbitMQ/MassTransit (Basket → Ordering).
- **Saga** triển khai cả hai biến thể: choreography (qua event bus) và orchestration (Saga.Orc + state machine + HTTP compensation).
- **Resilience** với Polly (Retry, CircuitBreaker, Timeout) trên HttpClient và gRPC client.
- **Observability**: HealthChecks UI cho liveness + WebHealthStatus tổng hợp; log tập trung về Elasticsearch + Kibana.

- Đóng gói **Docker Compose** cho dev và **Kubernetes** cho cluster (~57 resource Kustomize: StatefulSet cho DB, Deployment + HPA cho service, Ingress định tuyến 7 host).

6.2. Hạn chế

- **Chưa có distributed tracing**: dự án không tích hợp Jaeger/OpenTelemetry, khó debug độ trễ giữa nhiều service.
- **Chưa có rate limiting** ở gateway (cấu hình Ocelot RateLimit không bật).
- **Saga.Orc bypass gateway**: gọi trực tiếp localhost:6004/6005/6006 thay vì qua gateway.
- **Frontend dùng ROPC**: chấp nhận được cho demo nhưng không phải best practice (nên dùng Authorization Code + PKCE).
- **Eventual consistency** chưa được trình bày rõ ở phía UI: vì checkout trả 202, user có thể nhìn thấy "không có đơn nào" trong vài giây trước khi Order consumer tạo xong.
- **Phạm vi k8s dừng ở một máy host (Docker Desktop)**: HPA scale replicas nhưng pod không phân bố qua nhiều máy vật lý. Chưa demo được self-healing cross-node hay phân tải qua nhiều worker (cần Linux node thật/ảo, ngoài tầm phòng máy hiện có) — chi tiết ở §4.5.9.

6.3. Hướng phát triển

- Tích hợp **OpenTelemetry** + **Jaeger** để có distributed tracing end-to-end khi pod nằm trên nhiều máy.
- Bật **rate limiting** + **API key** ở Ocelot.
- Mở rộng phạm vi triển khai k8s thành **multi-node thật** khi có máy Linux (vật lý hoặc VM Multipass/Hyper-V), bổ sung MetalLB + NFS provisioner + image registry cục bộ để demo distributed scheduling cross-node.
- Chuyển manifest sang **Helm chart** + tích hợp **Prometheus** + **Grafana** để quan sát metric theo pod/service.

- Cấu hình thêm **Outbox pattern** cho Basket → đảm bảo publish event RabbitMQ là transactional với write Redis.
- Bổ sung **Read Model** riêng cho query đơn hàng (CQRS đầy đủ): denormalize sang Elasticsearch để search nhanh.
- Frontend chuyển sang **OAuth2 Authorization Code + PKCE** cho an toàn hơn.

6.4. Kết luận

Đề tài đã xây dựng một hệ thống thương mại điện tử thể hiện đầy đủ các pattern đặc trưng của hệ thống phân tán: tách microservice theo bounded context, API Gateway, ba mô hình giao tiếp REST + gRPC + Event, Saga ở cả hai biến thể, polyglot persistence, resilience pattern, container hoá và điều phối bằng Kubernetes. Phạm vi triển khai dừng ở cluster đơn-máy trên Docker Desktop — đủ để các pattern Kubernetes (declarative config, scheduling, scale-out, self-healing, rolling update) trực quan trong khuôn khổ học tập, đồng thời chỉ ra rõ những đánh đổi thực tế (eventual consistency, partial failure, vận hành đa dịch vụ) mà tài liệu phân tán mô tả.

TÀI LIỆU THAM KHẢO

1. Sam Newman, *Building Microservices: Designing Fine-Grained Systems*, O'Reilly, 2nd ed., 2021.
2. Chris Richardson, *Microservices Patterns: With examples in Java*, Manning, 2018.
3. Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.
4. Microsoft, *.NET Microservices: Architecture for Containerized .NET Applications*, eBook 2023. <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/>
5. Dick Hardt, *The OAuth 2.0 Authorization Framework*, IETF RFC 6749, 2012.
6. Michael Jones et al., *JSON Web Token (JWT)*, IETF RFC 7519, 2015.
7. OpenID Foundation, *OpenID Connect Core 1.0*, 2014.

8. Microsoft, *gRPC for .NET*. <https://learn.microsoft.com/en-us/aspnet/core/grpc/>
9. MassTransit Documentation. <https://masstransit.io/>
10. Duende Software, *Duende IdentityServer Documentation*. <https://docs.duendesoftware.com/identityserver/v7>
11. Ocelot Documentation. <https://ocelot.readthedocs.io/>
12. Polly: Resilience and transient-fault-handling library. <https://github.com/App-vNext/Polly>
13. Martin Fowler, *Polyglot Persistence*, 2011. <https://martinfowler.com/bliki/PolyglotPersistence.html>
14. Hangfire Documentation. <https://docs.hangfire.io/>
15. Kubernetes Authors, *Kubernetes Documentation*. <https://kubernetes.io/docs/>
16. Brendan Burns, Joe Beda, Kelsey Hightower, *Kubernetes: Up and Running*, O'Reilly, 3rd ed., 2022.
17. Marko Lukša, *Kubernetes in Action*, Manning, 2nd ed., 2023.
18. Kustomize Documentation. <https://kubectl.docs.kubernetes.io/references/kustomize/>
19. NGINX Ingress Controller Documentation. <https://kubernetes.github.io/ingress-nginx/>